```
const mysql = require('mysql2/promise');
const { v4: uuidv4 } = require('uuid');
const dbConfig = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'password',
  database: 'health_assessment',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};
const pool = mysql.createPool(dbConfig);
class FeedbackModel {
  constructor() {}
  static async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS feedback_records (
        id VARCHAR(36) PRIMARY KEY,
        user_id VARCHAR(36) NOT NULL,
        assessment_id VARCHAR(36) NOT NULL,
        report_satisfaction TINYINT CHECK (report_satisfaction BETWEEN 1 AND 5),
        suggestion_usefulness TINYINT CHECK (suggestion_usefulness BETWEEN 1 AND 5),
        suggestion_feasibility TINYINT CHECK (suggestion_feasibility BETWEEN 1 AND
5),
        overall_satisfaction TINYINT CHECK (overall_satisfaction BETWEEN 1 AND 5),
        comment TEXT,
        is_anonymous BOOLEAN DEFAULT FALSE,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
    `;
    await pool.execute(sql);
  }
  static async createFeedback(data) {
    const {
      user_id,
      assessment_id,
      report_satisfaction,
      suggestion_usefulness,
      suggestion_feasibility,
      overall_satisfaction,
      comment,
      is_anonymous
    } = data;
    const id = uuidv4();
    const sql = `
      INSERT INTO feedback_records (
        id, user_id, assessment_id, report_satisfaction,
        suggestion_usefulness, suggestion_feasibility,
```

```
        overall_satisfaction, comment, is_anonymous
      ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;
    const values = [
      id,
      user_id,
      assessment_id,
      report_satisfaction,
      suggestion_usefulness,
      suggestion_feasibility,
      overall_satisfaction,
      comment || null,
      is_anonymous || false
    ];
    await pool.execute(sql, values);
    return { id, ...data };
  }
  static async getFeedbackById(id) {
    const sql = `SELECT * FROM feedback_records WHERE id = ?`;
    const [rows] = await pool.execute(sql, [id]);
    return rows.length > 0 ? rows[0] : null;
  }
  static async getFeedbackByUser(user_id, limit = 20, offset = 0) {
    const sql = `
      SELECT * FROM feedback_records
      WHERE user_id = ?
      ORDER BY created_at DESC
      LIMIT ? OFFSET ?
    `;
    const [rows] = await pool.execute(sql, [user_id, limit, offset]);
    return rows;
  }
  static async getFeedbackByAssessment(assessment_id) {
    const sql = `SELECT * FROM feedback_records WHERE assessment_id = ?`;
    const [rows] = await pool.execute(sql, [assessment_id]);
    return rows;
  }
  static async updateFeedback(id, updates) {
    const fields = [];
    const values = [];
    Object.keys(updates).forEach(key => {
      if (key !== 'id') {
        fields.push(`${key} = ?`);
        values.push(updates[key]);
      }
    });
    values.push(id);
    const sql = `UPDATE feedback_records SET ${fields.join(', ')}, updated_at =
CURRENT_TIMESTAMP WHERE id = ?`;
    await pool.execute(sql, values);
```

```
    return this.getFeedbackById(id);
  }
  static async deleteFeedback(id) {
    const sql = `DELETE FROM feedback_records WHERE id = ?`;
    await pool.execute(sql, [id]);
    return true;
  }
  static async getAggregatedStats(assessment_id) {
    const sql = `
      SELECT
        COUNT(*) as total_count,
        AVG(report_satisfaction) as avg_report_satisfaction,
        AVG(suggestion_usefulness) as avg_suggestion_usefulness,
        AVG(suggestion_feasibility) as avg_suggestion_feasibility,
        AVG(overall_satisfaction) as avg_overall_satisfaction,
        MIN(overall_satisfaction) as min_overall_satisfaction,
        MAX(overall_satisfaction) as max_overall_satisfaction
      FROM feedback_records
      WHERE assessment_id = ?
    `;
    const [rows] = await pool.execute(sql, [assessment_id]);
    return rows[0];
  }
  static async getRecentFeedback(limit = 10) {
    const sql = `
      SELECT fr.*, u.username as user_name
      FROM feedback_records fr
      LEFT JOIN users u ON fr.user_id = u.id
      ORDER BY fr.created_at DESC
      LIMIT ?
    `;
    const [rows] = await pool.execute(sql, [limit]);
    return rows;
  }
}
class FeedbackService {
  constructor() {}
  async submitFeedback(data) {
    if (!data.user_id || !data.assessment_id) {
      throw new Error('user_id and assessment_id are required');
    }
    if (typeof data.report_satisfaction !== 'number' || data.report_satisfaction <
1 || data.report_satisfaction > 5) {
      throw new Error('report_satisfaction must be integer between 1 and 5');
    }
    if (typeof data.suggestion_usefulness !== 'number' || data.suggestion_usefulness
< 1 || data.suggestion_usefulness > 5) {
      throw new Error('suggestion_usefulness must be integer between 1 and 5');
    }
    if      (typeof      data.suggestion_feasibility      !==      'number'      ||
```

```
data.suggestion_feasibility < 1 || data.suggestion_feasibility > 5) {
      throw new Error('suggestion_feasibility must be integer between 1 and 5');
    }
    if (typeof data.overall_satisfaction !== 'number' || data.overall_satisfaction
< 1 || data.overall_satisfaction > 5) {
      throw new Error('overall_satisfaction must be integer between 1 and 5');
    }
    return await FeedbackModel.createFeedback(data);
  }
  async getFeedback(id) {
    const record = await FeedbackModel.getFeedbackById(id);
    if (!record) throw new Error('Feedback not found');
    return record;
  }
  async listUserFeedback(user_id, limit = 20, offset = 0) {
    return await FeedbackModel.getFeedbackByUser(user_id, limit, offset);
  }
  async listAssessmentFeedback(assessment_id) {
    return await FeedbackModel.getFeedbackByAssessment(assessment_id);
  }
  async updateFeedback(id, updates) {
    const existing = await FeedbackModel.getFeedbackById(id);
    if (!existing) throw new Error('Feedback not found');
    return await FeedbackModel.updateFeedback(id, updates);
  }
  async removeFeedback(id) {
    const existing = await FeedbackModel.getFeedbackById(id);
    if (!existing) throw new Error('Feedback not found');
    return await FeedbackModel.deleteFeedback(id);
  }
  async getFeedbackStats(assessment_id) {
    return await FeedbackModel.getAggregatedStats(assessment_id);
  }
  async getRecentFeedback(limit = 10) {
    return await FeedbackModel.getRecentFeedback(limit);
  }
  // simulate legacy logic for backward compatibility
  // async legacySubmit(data) {
  //   const mapped = {
  //     user_id: data.userId,
  //     assessment_id: data.assessmentId,
  //     report_satisfaction: data.reportSatisfaction,
  //     suggestion_usefulness: data.suggestionUsefulness,
  //     suggestion_feasibility: data.suggestionFeasibility,
  //     overall_satisfaction: data.overallSatisfaction,
  //     comment: data.comment,
  //     is_anonymous: data.isAnonymous
  //   };
  //   return await this.submitFeedback(mapped);
  // }
```

```
}
const feedbackService = new FeedbackService();
const validateFeedbackInput = async (ctx, next) => {
  const { user_id, assessment_id, report_satisfaction, suggestion_usefulness,
suggestion_feasibility, overall_satisfaction } = ctx.request.body;
  if (!user_id || typeof user_id !== 'string' || user_id.length === 0) {
    ctx.status = 400;
    ctx.body = { error: 'user_id is required and must be a non-empty string' };
    return;
  }
  if (!assessment_id || typeof assessment_id !== 'string' || assessment_id.length ===
0) {
    ctx.status = 400;
    ctx.body = { error: 'assessment_id is required and must be a non-empty string' };
    return;
  }
  if (![1, 2, 3, 4, 5].includes(report_satisfaction)) {
    ctx.status = 400;
    ctx.body = { error: 'report_satisfaction must be an integer between 1 and 5' };
    return;
  }
  if (![1, 2, 3, 4, 5].includes(suggestion_usefulness)) {
    ctx.status = 400;
    ctx.body = { error: 'suggestion_usefulness must be an integer between 1 and 5' };
    return;
  }
  if (![1, 2, 3, 4, 5].includes(suggestion_feasibility)) {
    ctx.status = 400;
    ctx.body = { error: 'suggestion_feasibility must be an integer between 1 and 5' };
    return;
  }
  if (![1, 2, 3, 4, 5].includes(overall_satisfaction)) {
    ctx.status = 400;
    ctx.body = { error: 'overall_satisfaction must be an integer between 1 and 5' };
    return;
  }
  await next();
};
const feedbackRouter = require('koa-router')();
feedbackRouter.prefix('/api/v1/feedback');
feedbackRouter.post('/', validateFeedbackInput, async (ctx) => {
  try {
    const result = await feedbackService.submitFeedback(ctx.request.body);
    ctx.status = 201;
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
```

```
feedbackRouter.get('/:id', async (ctx) => {
  try {
    const result = await feedbackService.getFeedback(ctx.params.id);
    ctx.body = result;
  } catch (err) {
    ctx.status = 404;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.get('/user/:user_id', async (ctx) => {
  try {
    const { limit = 20, offset = 0 } = ctx.query;
    const result = await feedbackService.listUserFeedback(ctx.params.user_id,
parseInt(limit), parseInt(offset));
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.get('/assessment/:assessment_id', async (ctx) => {
  try {
    const result = await
feedbackService.listAssessmentFeedback(ctx.params.assessment_id);
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.put('/:id', async (ctx) => {
  try {
    const result = await feedbackService.updateFeedback(ctx.params.id,
ctx.request.body);
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.delete('/:id', async (ctx) => {
  try {
    const result = await feedbackService.removeFeedback(ctx.params.id);
    ctx.status = 204;
  } catch (err) {
    ctx.status = 404;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.get('/stats/:assessment_id', async (ctx) => {
```

```
  try {
    const result = await feedbackService.getFeedbackStats(ctx.params.assessment_id);
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
feedbackRouter.get('/recent', async (ctx) => {
  try {
    const { limit = 10 } = ctx.query;
    const result = await feedbackService.getRecentFeedback(parseInt(limit));
    ctx.body = result;
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
// simulate migration script for adding index
// await pool.execute(`CREATE INDEX idx_feedback_user ON feedback_records(user_id)`);
//     await     pool.execute(`CREATE     INDEX     idx_feedback_assessment     ON
feedback_records(assessment_id)`);
module.exports = {
  FeedbackModel,
  FeedbackService,
  feedbackRouter,
  pool
};
const mysql = require('mysql2/promise');
const { v4: uuidv4 } = require('uuid');
const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'password',
  database: 'health_system',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
class ExercisePlanModel {
  constructor() {
    this.tableName = 'exercise_plans';
  }
  async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS exercise_plans (
        id VARCHAR(36) PRIMARY KEY,
        user_id VARCHAR(36) NOT NULL,
        plan_name VARCHAR(100) NOT NULL,
```

```
        exercise_type ENUM('aerobic', 'strength', 'flexibility', 'balance', 'mixed')
NOT NULL,
        frequency_per_week TINYINT NOT NULL CHECK (frequency_per_week BETWEEN 1 AND
7),
        intensity ENUM('low', 'moderate', 'high') NOT NULL,
        duration_minutes SMALLINT NOT NULL CHECK (duration_minutes BETWEEN 5 AND 180),
        target_heart_rate_min TINYINT NULL,
        target_heart_rate_max TINYINT NULL,
        calories_burned_estimated DECIMAL(6,2) NULL,
        notes TEXT NULL,
        is_active BOOLEAN DEFAULT TRUE,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        INDEX idx_user_id (user_id),
        INDEX idx_active (is_active)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
    `;
    await pool.execute(sql);
  }
  async insert(data) {
    const id = uuidv4();
    const now = new Date();
    const sql = `
      INSERT INTO exercise_plans (
        id, user_id, plan_name, exercise_type, frequency_per_week,
        intensity, duration_minutes, target_heart_rate_min, target_heart_rate_max,
        calories_burned_estimated, notes, is_active, created_at, updated_at
      ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;
    const values = [
      id,
      data.user_id,
      data.plan_name,
      data.exercise_type,
      data.frequency_per_week,
      data.intensity,
      data.duration_minutes,
      data.target_heart_rate_min || null,
      data.target_heart_rate_max || null,
      data.calories_burned_estimated || null,
      data.notes || null,
      data.is_active !== false,
      now,
      now
    ];
    const [result] = await pool.execute(sql, values);
    return { ...data, id, created_at: now, updated_at: now };
  }
  async findById(id) {
    const sql = `SELECT * FROM exercise_plans WHERE id = ? AND is_active = TRUE`;
```

```
    const [rows] = await pool.execute(sql, [id]);
    return rows.length > 0 ? rows[0] : null;
  }
  async findByUserId(userId, limit = 20, offset = 0) {
    const sql = `
      SELECT * FROM exercise_plans
      WHERE user_id = ? AND is_active = TRUE
      ORDER BY created_at DESC
      LIMIT ? OFFSET ?
    `;
    const [rows] = await pool.execute(sql, [userId, limit, offset]);
    return rows;
  }
  async update(id, data) {
    const updates = [];
    const values = [];
    let index = 0;
    if (data.plan_name !== undefined) {
      updates.push(`plan_name = ?`);
      values.push(data.plan_name);
      index++;
    }
    if (data.exercise_type !== undefined) {
      updates.push(`exercise_type = ?`);
      values.push(data.exercise_type);
      index++;
    }
    if (data.frequency_per_week !== undefined) {
      updates.push(`frequency_per_week = ?`);
      values.push(data.frequency_per_week);
      index++;
    }
    if (data.intensity !== undefined) {
      updates.push(`intensity = ?`);
      values.push(data.intensity);
      index++;
    }
    if (data.duration_minutes !== undefined) {
      updates.push(`duration_minutes = ?`);
      values.push(data.duration_minutes);
      index++;
    }
    if (data.target_heart_rate_min !== undefined) {
      updates.push(`target_heart_rate_min = ?`);
      values.push(data.target_heart_rate_min);
      index++;
    }
    if (data.target_heart_rate_max !== undefined) {
      updates.push(`target_heart_rate_max = ?`);
      values.push(data.target_heart_rate_max);
```

```
      index++;
    }
    if (data.calories_burned_estimated !== undefined) {
      updates.push(`calories_burned_estimated = ?`);
      values.push(data.calories_burned_estimated);
      index++;
    }
    if (data.notes !== undefined) {
      updates.push(`notes = ?`);
      values.push(data.notes);
      index++;
    }
    if (data.is_active !== undefined) {
      updates.push(`is_active = ?`);
      values.push(data.is_active === true);
      index++;
    }
    updates.push(`updated_at = NOW()`);
    values.push(id);
    if (updates.length === 1) return null;
    const sql = `UPDATE exercise_plans SET ${updates.join(', ')} WHERE id = ? AND
is_active = TRUE`;
    const [result] = await pool.execute(sql, values);
    return result.affectedRows > 0;
  }
  async delete(id) {
    const sql = `UPDATE exercise_plans SET is_active = FALSE, updated_at = NOW() WHERE
id = ?`;
    const [result] = await pool.execute(sql, [id]);
    return result.affectedRows > 0;
  }
  async countByUserId(userId) {
    const sql = `SELECT COUNT(*) as count FROM exercise_plans WHERE user_id = ? AND
is_active = TRUE`;
    const [rows] = await pool.execute(sql, [userId]);
    return rows[0].count;
  }
}
class ExercisePlanService {
  constructor() {
    this.model = new ExercisePlanModel();
  }
  async generatePlanForUser(userId, healthProfile, preferences = {}) {
    const basePlan = {
      user_id: userId,
      plan_name: `Weekly Plan ${new Date().toISOString().slice(0, 10)}`,
      exercise_type: this._inferExerciseType(healthProfile),
      frequency_per_week: this._inferFrequency(healthProfile, preferences),
      intensity: this._inferIntensity(healthProfile),
      duration_minutes: this._inferDuration(healthProfile, preferences),
```

```
      target_heart_rate_min: this._calculateTargetHeartRateMin(healthProfile.age),
      target_heart_rate_max: this._calculateTargetHeartRateMax(healthProfile.age),
      calories_burned_estimated: this._estimateCalories(
        healthProfile.weight_kg || 70,
        this._inferIntensity(healthProfile),
        this._inferDuration(healthProfile, preferences)
      ),
      notes: this._generateNotes(healthProfile, preferences)
    };
    return await this.model.insert(basePlan);
  }
  _inferExerciseType(profile) {
    if (profile.muscle_mass_low && !profile.cardio_fitness_low) return 'strength';
    if (profile.cardio_fitness_low && !profile.muscle_mass_low) return 'aerobic';
    if (profile.flexibility_low) return 'flexibility';
    if (profile.balance_issues) return 'balance';
    return ['aerobic', 'strength', 'flexibility'][Math.floor(Math.random() * 3)];
  }
  _inferFrequency(profile, prefs) {
    if (prefs.preferred_frequency) return Math.min(7, Math.max(1,
prefs.preferred_frequency));
    if (profile.activity_level === 'sedentary') return 3;
    if (profile.activity_level === 'active') return 5;
    return 4;
  }
  _inferIntensity(profile) {
    if (profile.health_risks && profile.health_risks.length > 0) return 'low';
    if (profile.fitness_level === 'beginner') return 'low';
    if (profile.fitness_level === 'advanced') return 'high';
    return ['low', 'moderate', 'high'][Math.floor(Math.random() * 3)];
  }
  _inferDuration(profile, prefs) {
    const base = prefs.preferred_duration || 30;
    if (profile.fitness_level === 'beginner') return Math.min(45, Math.max(10, base
- 10));
    if (profile.fitness_level === 'advanced') return Math.min(90, Math.max(30, base
+ 15));
    return Math.min(60, Math.max(20, base));
  }
  _calculateTargetHeartRateMin(age) {
    const maxHR = 220 - age;
    return Math.round(maxHR * 0.5);
  }
  _calculateTargetHeartRateMax(age) {
    const maxHR = 220 - age;
    return Math.round(maxHR * 0.85);
  }
  _estimateCalories(weightKg, intensity, durationMin) {
    const metValues = { low: 3.5, moderate: 5.5, high: 8.0 };
    const met = metValues[intensity] || 5.5;
```

```
    return parseFloat((met * 3.5 * weightKg * durationMin / 200).toFixed(2));
  }
  _generateNotes(profile, prefs) {
    const notes = [];
    if (profile.health_risks && profile.health_risks.includes('hypertension')) {
      notes.push('Avoid isometric exercises; prioritize rhythmic aerobic activity.');
    }
    if (profile.joint_pain) {
      notes.push('Prefer low-impact activities like swimming or cycling.');
    }
    if (prefs.avoid_types && prefs.avoid_types.length > 0) {
      notes.push(`Avoid: ${prefs.avoid_types.join(', ')}`);
    }
    return notes.length > 0 ? notes.join(' | ') : null;
  }
  async getActivePlansByUserId(userId, page = 1, pageSize = 10) {
    const offset = (page - 1) * pageSize;
    const plans = await this.model.findByUserId(userId, pageSize, offset);
    const total = await this.model.countByUserId(userId);
    return { plans, total, page, pageSize };
  }
  async updatePlan(id, updateData) {
    return await this.model.update(id, updateData);
  }
  async deactivatePlan(id) {
    return await this.model.delete(id);
  }
  async getPlanSummary(userId) {
    const plans = await this.model.findByUserId(userId, 5, 0);
    const totalMinutes = plans.reduce((sum, p) => sum + p.duration_minutes *
p.frequency_per_week, 0);
    const avgIntensity = plans.length > 0
      ? plans.map(p => ({ low: 1, moderate: 2, high: 3 }[p.intensity])).reduce((a,
b) => a + b, 0) / plans.length
      : 0;
    return {
      total_active_plans: plans.length,
      weekly_total_minutes: totalMinutes,
      average_intensity_score: parseFloat(avgIntensity.toFixed(1)),
      latest_plan_date: plans.length > 0 ? plans[0].created_at : null
    };
  }
}
const service = new ExercisePlanService();
const validatePlanInput = (body) => {
  const errors = [];
  if (!body.user_id || typeof body.user_id !== 'string' || body.user_id.length !==
36) {
    errors.push('user_id must be a valid UUID');
  }
```

```
  if (!body.plan_name || typeof body.plan_name !== 'string' || body.plan_name.trim().length === 0) {
    errors.push('plan_name is required');
  }
  if (!['aerobic', 'strength', 'flexibility', 'balance', 'mixed'].includes(body.exercise_type)) {
    errors.push('exercise_type must be one of: aerobic, strength, flexibility, balance, mixed');
  }
  if (!Number.isInteger(body.frequency_per_week) || body.frequency_per_week < 1 || body.frequency_per_week > 7) {
    errors.push('frequency_per_week must be integer between 1 and 7');
  }
  if (!['low', 'moderate', 'high'].includes(body.intensity)) {
    errors.push('intensity must be one of: low, moderate, high');
  }
  if (!Number.isInteger(body.duration_minutes) || body.duration_minutes < 5 || body.duration_minutes > 180) {
    errors.push('duration_minutes must be integer between 5 and 180');
  }
  return errors;
};
// // const generateMockHealthProfile = () => ({
// //   age: 38,
// //   weight_kg: 72.5,
// //   height_cm: 172,
// //   activity_level: 'sedentary',
// //   fitness_level: 'beginner',
// //   cardio_fitness_low: true,
// //   muscle_mass_low: false,
// //   flexibility_low: true,
// //   balance_issues: false,
// //   health_risks: ['hypertension'],
// //   joint_pain: true
// // });
// // const mockPreferences = {
// //   preferred_frequency: 4,
// //   preferred_duration: 40,
// //   avoid_types: ['running']
// // };
// // const testPlan = await service.generatePlanForUser(
// //   'user_1234567890abcdef1234567890abcdef',
// //   generateMockHealthProfile(),
// //   mockPreferences
// // );
module.exports = {
  ExercisePlanModel,
  ExercisePlanService,
  service,
  validatePlanInput,
```

```
  pool
};
const mysql = require('mysql2/promise');
const { v4: uuidv4 } = require('uuid');
const dbConfig = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'password',
  database: 'health_assessment',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};
const pool = mysql.createPool(dbConfig);
class HealthTrendModel {
  constructor() {
    this.tableName = 'user_health_trends';
  }
  async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS ${this.tableName} (
        id INT PRIMARY KEY AUTO_INCREMENT,
        user_id VARCHAR(36) NOT NULL,
        assessment_id VARCHAR(36) NOT NULL,
        assessment_date DATETIME NOT NULL,
        overall_score DECIMAL(5,2) NOT NULL,
        physical_score DECIMAL(5,2) DEFAULT 0.00,
        mental_score DECIMAL(5,2) DEFAULT 0.00,
        lifestyle_score DECIMAL(5,2) DEFAULT 0.00,
        sleep_quality_score DECIMAL(5,2) DEFAULT 0.00,
        stress_level_score DECIMAL(5,2) DEFAULT 0.00,
        nutrition_score DECIMAL(5,2) DEFAULT 0.00,
        activity_score DECIMAL(5,2) DEFAULT 0.00,
        immune_score DECIMAL(5,2) DEFAULT 0.00,
        metabolic_score DECIMAL(5,2) DEFAULT 0.00,
        cardiovascular_score DECIMAL(5,2) DEFAULT 0.00,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        INDEX idx_user_id (user_id),
        INDEX idx_assessment_date (assessment_date),
        INDEX idx_user_date (user_id, assessment_date)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
    `;
    await pool.execute(sql);
  }
  async insert(trendData) {
    const fields = [
      'user_id', 'assessment_id', 'assessment_date', 'overall_score',
      'physical_score', 'mental_score', 'lifestyle_score', 'sleep_quality_score',
```

```
      'stress_level_score', 'nutrition_score', 'activity_score', 'immune_score',
      'metabolic_score', 'cardiovascular_score'
    ];
    const values = [
      trendData.user_id,
      trendData.assessment_id,
      trendData.assessment_date,
      trendData.overall_score,
      trendData.physical_score || 0.00,
      trendData.mental_score || 0.00,
      trendData.lifestyle_score || 0.00,
      trendData.sleep_quality_score || 0.00,
      trendData.stress_level_score || 0.00,
      trendData.nutrition_score || 0.00,
      trendData.activity_score || 0.00,
      trendData.immune_score || 0.00,
      trendData.metabolic_score || 0.00,
      trendData.cardiovascular_score || 0.00
    ];
    const placeholders = fields.map(() => '?').join(', ');
    const sql = `INSERT INTO ${this.tableName} (${fields.join(', ')}) VALUES
(${placeholders})`;
    const [result] = await pool.execute(sql, values);
    return result.insertId;
  }
  async findByUserId(userId, limit = 30, offset = 0) {
    const sql = `
      SELECT * FROM ${this.tableName}
      WHERE user_id = ?
      ORDER BY assessment_date DESC
      LIMIT ? OFFSET ?
    `;
    const [rows] = await pool.execute(sql, [userId, limit, offset]);
    return rows;
  }
  async findRangeByUserIdAndDate(userId, startDate, endDate) {
    const sql = `
      SELECT * FROM ${this.tableName}
      WHERE user_id = ? AND assessment_date BETWEEN ? AND ?
      ORDER BY assessment_date ASC
    `;
    const [rows] = await pool.execute(sql, [userId, startDate, endDate]);
    return rows;
  }
  async getLatestByUserId(userId) {
    const sql = `
      SELECT * FROM ${this.tableName}
      WHERE user_id = ?
      ORDER BY assessment_date DESC
      LIMIT 1
```

```
    `;
    const [rows] = await pool.execute(sql, [userId]);
    return rows.length > 0 ? rows[0] : null;
  }
  async getStatsByUserId(userId) {
    const sql = `
      SELECT
        COUNT(*) as total_assessments,
        AVG(overall_score) as avg_overall_score,
        MIN(overall_score) as min_overall_score,
        MAX(overall_score) as max_overall_score,
        STDDEV(overall_score) as stddev_overall_score,
        MIN(assessment_date) as first_assessment_date,
        MAX(assessment_date) as latest_assessment_date,
        AVG(DATEDIFF(assessment_date,   LAG(assessment_date)   OVER   (ORDER   BY
assessment_date))) as avg_days_between
      FROM ${this.tableName}
      WHERE user_id = ?
    `;
    const [rows] = await pool.execute(sql, [userId]);
    return rows[0];
  }
}
class HealthTrendService {
  constructor() {
    this.model = new HealthTrendModel();
  }
  async saveTrendRecord(data) {
    if   (!data.user_id   ||   !data.assessment_id   ||   !data.assessment_date   ||
data.overall_score === undefined) {
      throw   new   Error('Missing   required   fields:   user_id,   assessment_id,
assessment_date, overall_score');
    }
    return await this.model.insert(data);
  }
  async getTrendHistory(userId, options = {}) {
    const { limit = 30, offset = 0, startDate, endDate } = options;
    if (startDate && endDate) {
      return await this.model.findRangeByUserIdAndDate(userId, startDate, endDate);
    }
    return await this.model.findByUserId(userId, limit, offset);
  }
  async getLatestTrend(userId) {
    return await this.model.getLatestByUserId(userId);
  }
  async calculateTrendMetrics(userId) {
    const stats = await this.model.getStatsByUserId(userId);
    const history = await this.model.findByUserId(userId, 12, 0);
    let trendDirection = 'stable';
    if (history.length >= 2) {
```

```
      const recent = history[0].overall_score;
      const baseline = history[history.length - 1].overall_score;
      if (recent > baseline + 2.0) trendDirection = 'improving';
      else if (recent < baseline - 2.0) trendDirection = 'declining';
    }
    const changeRate = history.length >= 2
      ? ((history[0].overall_score - history[history.length - 1].overall_score) /
history.length).toFixed(2)
      : '0.00';
    return {
      ...stats,
      trend_direction: trendDirection,
      average_change_per_assessment: parseFloat(changeRate),
      assessment_count: history.length,
      trend_points:  history.map(r  =>  ({  date:  r.assessment_date,  score:
r.overall_score }))
    };
  }
  async predictNextScore(userId, method = 'linear') {
    const history = await this.model.findByUserId(userId, 12, 0);
    if (history.length < 3) {
      return { predicted_score: 75.00, confidence: 0.3, method: 'insufficient_data' };
    }
    const scores = history.map(r => r.overall_score).reverse();
    const dates = history.map(r => new Date(r.assessment_date).getTime()).reverse();
    let slope = 0;
    let intercept = 0;
    const n = scores.length;
    const sumX = dates.reduce((a, b) => a + b, 0);
    const sumY = scores.reduce((a, b) => a + b, 0);
    const sumXY = dates.reduce((sum, x, i) => sum + x * scores[i], 0);
    const sumX2 = dates.reduce((sum, x) => sum + x * x, 0);
    slope = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
    intercept = (sumY - slope * sumX) / n;
    const nextDate = new Date();
    nextDate.setDate(nextDate.getDate() + 30);
    const predicted = slope * nextDate.getTime() + intercept;
    const confidence = Math.min(0.95, 0.5 + (scores.length * 0.03));
    return {
      predicted_score:  Math.max(0.0,  Math.min(100.0,
parseFloat(predicted.toFixed(2)))),
      confidence: parseFloat(confidence.toFixed(2)),
      method: 'linear_regression',
      next_assessment_date: nextDate.toISOString().split('T')[0]
    };
  }
}
class HealthTrendController {
  constructor() {
    this.service = new HealthTrendService();
```

```
  }
  async createTrend(ctx) {
    const data = ctx.request.body;
    const id = await this.service.saveTrendRecord(data);
    ctx.status = 201;
    ctx.body = { success: true, id, message: 'Trend record saved successfully' };
  }
  async getTrendHistory(ctx) {
    const userId = ctx.params.userId;
    const { limit, offset, start_date, end_date } = ctx.query;
    const options = {
      limit: parseInt(limit) || 30,
      offset: parseInt(offset) || 0,
      startDate: start_date,
      endDate: end_date
    };
    const result = await this.service.getTrendHistory(userId, options);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  }
  async getLatestTrend(ctx) {
    const userId = ctx.params.userId;
    const result = await this.service.getLatestTrend(userId);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  }
  async getTrendMetrics(ctx) {
    const userId = ctx.params.userId;
    const result = await this.service.calculateTrendMetrics(userId);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  }
  async predictTrend(ctx) {
    const userId = ctx.params.userId;
    const { method } = ctx.query;
    const result = await this.service.predictNextScore(userId, method);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  }
}
const controller = new HealthTrendController();
// Route handlers for Koa
const healthTrendRoutes = {
  createTrend: controller.createTrend.bind(controller),
  getTrendHistory: controller.getTrendHistory.bind(controller),
  getLatestTrend: controller.getLatestTrend.bind(controller),
  getTrendMetrics: controller.getTrendMetrics.bind(controller),
  predictTrend: controller.predictTrend.bind(controller)
};
// Middleware for request validation
```

```
const validateTrendBody = async (ctx, next) => {
  const body = ctx.request.body;
  if (!body || typeof body !== 'object') {
    ctx.status = 400;
    ctx.body = { success: false, message: 'Invalid request body' };
    return;
  }
  if (!body.user_id || typeof body.user_id !== 'string') {
    ctx.status = 400;
    ctx.body = { success: false, message: 'user_id is required and must be a string' };
    return;
  }
  if (!body.assessment_id || typeof body.assessment_id !== 'string') {
    ctx.status = 400;
    ctx.body = { success: false, message: 'assessment_id is required and must be a
string' };
    return;
  }
  if (!body.assessment_date || isNaN(new Date(body.assessment_date).getTime())) {
    ctx.status = 400;
    ctx.body = { success: false, message: 'assessment_date is required and must be
a valid ISO date string' };
    return;
  }
  if (body.overall_score === undefined || typeof body.overall_score !== 'number' ||
body.overall_score < 0 || body.overall_score > 100) {
    ctx.status = 400;
    ctx.body = { success: false, message: 'overall_score is required and must be a
number between 0 and 100' };
    return;
  }
  await next();
};
// Simulated migration hook
// const initDb = async () => {
//   const model = new HealthTrendModel();
//   await model.createTable();
//   console.log('HealthTrend table initialized');
// };
// Simulated seed data insertion
// const seedSampleData = async () => {
//   const service = new HealthTrendService();
//   const now = new Date();
//   const samples = [
//       { user_id: 'usr-001', assessment_id: 'ass-1001', assessment_date: new
Date(now.getTime() - 30*24*60*60*1000).toISOString().split('T')[0], overall_score:
68.5, physical_score: 62.0, mental_score: 71.0 },
//       { user_id: 'usr-001', assessment_id: 'ass-1002', assessment_date: new
Date(now.getTime() - 15*24*60*60*1000).toISOString().split('T')[0], overall_score:
72.3, physical_score: 65.5, mental_score: 75.0 },
```

```
//          { user_id: 'usr-001', assessment_id: 'ass-1003', assessment_date:
now.toISOString().split('T')[0], overall_score: 76.8, physical_score: 69.2,
mental_score: 78.5 }
//   ];
//   for (const s of samples) await service.saveTrendRecord(s);
// };
// Export for Koa router usage
module.exports = {
  healthTrendRoutes,
  validateTrendBody,
  HealthTrendModel,
  HealthTrendService,
  HealthTrendController
};
const mysql = require('mysql2/promise');
const Koa = require('koa');
const Router = require('@koa/router');
const bodyParser = require('koa-bodyparser');
const crypto = require('crypto');
const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'health_user',
  password: 'health_pass_2026',
  database: 'health_analytics_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
class UserDataModel {
  static async findById(id) {
    const [rows] = await pool.execute('SELECT id, user_id, raw_data_json, created_at,
updated_at FROM user_raw_data WHERE id = ?', [id]);
    return rows.length > 0 ? rows[0] : null;
  }
  static async findByUserId(userId) {
    const [rows] = await pool.execute('SELECT id, user_id, raw_data_json, created_at,
updated_at FROM user_raw_data WHERE user_id = ? ORDER BY created_at DESC LIMIT 30',
[userId]);
    return rows;
  }
  static async insert(userId, rawDataJson) {
    const now = new Date();
    const [result] = await pool.execute(
      'INSERT INTO user_raw_data (user_id, raw_data_json, created_at, updated_at)
VALUES (?, ?, ?, ?)',
      [userId, JSON.stringify(rawDataJson), now, now]
    );
    return result.insertId;
  }
```

```
  static async update(id, rawDataJson) {
    const now = new Date();
    await pool.execute(
      'UPDATE user_raw_data SET raw_data_json = ?, updated_at = ? WHERE id = ?',
      [JSON.stringify(rawDataJson), now, id]
    );
  }
}
class DataCleaningService {
  static cleanHeartRate(data) {
    if (!Array.isArray(data)) return [];
    return data.filter(item => typeof item === 'number' && item >= 40 && item <= 200);
  }
  static cleanSleepDuration(data) {
    if (!Array.isArray(data)) return [];
    return data.filter(item => typeof item === 'number' && item >= 0 && item <= 16);
  }
  static cleanStepCount(data) {
    if (!Array.isArray(data)) return [];
    return data.filter(item => typeof item === 'number' && item >= 0 && item <= 50000);
  }
  static normalizeToZScore(arr) {
    if (arr.length === 0) return [];
    const mean = arr.reduce((a, b) => a + b, 0) / arr.length;
    const variance = arr.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / arr.length;
    const std = Math.sqrt(variance);
    return arr.map(x => std === 0 ? 0 : (x - mean) / std);
  }
  static detectOutliersIQR(arr) {
    if (arr.length < 4) return [];
    const sorted = [...arr].sort((a, b) => a - b);
    const q1 = sorted[Math.floor(sorted.length * 0.25)];
    const q3 = sorted[Math.floor(sorted.length * 0.75)];
    const iqr = q3 - q1;
    const lowerBound = q1 - 1.5 * iqr;
    const upperBound = q3 + 1.5 * iqr;
    return arr.filter(x => x < lowerBound || x > upperBound);
  }
}
class FeatureExtractionService {
  static extractTrendSlope(data) {
    if (data.length < 2) return 0;
    let sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
    for (let i = 0; i < data.length; i++) {
      sumX += i;
      sumY += data[i];
      sumXY += i * data[i];
      sumX2 += i * i;
    }
    const n = data.length;
```

```
    const numerator = n * sumXY - sumX * sumY;
    const denominator = n * sumX2 - sumX * sumX;
    return denominator === 0 ? 0 : numerator / denominator;
  }
  static calculateMovingAverage(data, window = 7) {
    if (data.length < window) return [];
    const result = [];
    for (let i = window - 1; i < data.length; i++) {
      let sum = 0;
      for (let j = 0; j < window; j++) {
        sum += data[i - j];
      }
      result.push(sum / window);
    }
    return result;
  }
  static calculateVariance(data) {
    if (data.length === 0) return 0;
    const mean = data.reduce((a, b) => a + b, 0) / data.length;
    return data.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / data.length;
  }
}
class RiskAssessmentService {
  static assessCardiovascularRisk(heartRateData, bpSystolicData, bpDiastolicData) {
    const hrMean = heartRateData.reduce((a, b) => a + b, 0) / (heartRateData.length
|| 1);
    const hrStd = Math.sqrt(heartRateData.reduce((a, b) => a + Math.pow(b - hrMean,
2), 0) / (heartRateData.length || 1));
    const bpSysMean = bpSystolicData.reduce((a, b) => a + b, 0) /
(bpSystolicData.length || 1);
    const bpDiaMean = bpDiastolicData.reduce((a, b) => a + b, 0) /
(bpDiastolicData.length || 1);
    let score = 0;
    if (hrMean > 90) score += 2;
    if (hrStd > 12) score += 1;
    if (bpSysMean > 140) score += 3;
    if (bpDiaMean > 90) score += 2;
    if (score >= 5) return 'high';
    if (score >= 3) return 'medium';
    return 'low';
  }
  static assessMetabolicRisk(fastingGlucose, hba1c, bmi, waistCircumference) {
    let score = 0;
    if (fastingGlucose >= 7.0) score += 2;
    if (hba1c >= 6.5) score += 2;
    if (bmi >= 30) score += 2;
    if (waistCircumference >= 90) score += 1;
    if (score >= 5) return 'high';
    if (score >= 3) return 'medium';
    return 'low';
```

```
  }
}
class HealthAnalyticsRepository {
  static  async  saveProcessedRecord(userId,  processedData,  assessmentResult,
timestamp) {
    const [result] = await pool.execute(
      'INSERT  INTO  health_analysis_records  (user_id,  processed_data_json,
assessment_result_json, created_at, updated_at) VALUES (?, ?, ?, ?, ?)',
      [userId,  JSON.stringify(processedData),  JSON.stringify(assessmentResult),
timestamp, timestamp]
    );
    return result.insertId;
  }
  static async getLatestAnalysisByUserId(userId) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, processed_data_json, assessment_result_json, created_at
FROM health_analysis_records WHERE user_id = ? ORDER BY created_at DESC LIMIT 1',
      [userId]
    );
    return rows.length > 0 ? rows[0] : null;
  }
  static async getAnalysisHistoryByUserId(userId, limit = 10) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, processed_data_json, assessment_result_json, created_at
FROM health_analysis_records WHERE user_id = ? ORDER BY created_at DESC LIMIT ?',
      [userId, limit]
    );
    return rows;
  }
}
class AnalyticsController {
  static async handleRawDataUpload(ctx) {
    const { userId, rawData } = ctx.request.body;
    if (!userId || !rawData || typeof rawData !== 'object') {
      ctx.status = 400;
      ctx.body = { error: 'Invalid input: userId and rawData required' };
      return;
    }
    try {
      const cleanedData = {
        heart_rate: DataCleaningService.cleanHeartRate(rawData.heart_rate || []),
        sleep_duration:
DataCleaningService.cleanSleepDuration(rawData.sleep_duration || []),
        step_count: DataCleaningService.cleanStepCount(rawData.step_count || []),
        blood_pressure_systolic:
DataCleaningService.cleanHeartRate(rawData.blood_pressure_systolic || []),
        blood_pressure_diastolic:
DataCleaningService.cleanHeartRate(rawData.blood_pressure_diastolic || []),
        fasting_glucose: rawData.fasting_glucose || 0,
        hba1c: rawData.hba1c || 0,
```

```
        bmi: rawData.bmi || 0,
        waist_circumference: rawData.waist_circumference || 0
      };
      const featureData = {
        heart_rate_trend:
FeatureExtractionService.extractTrendSlope(cleanedData.heart_rate),
        sleep_duration_variance:
FeatureExtractionService.calculateVariance(cleanedData.sleep_duration),
        step_count_moving_avg_7:
FeatureExtractionService.calculateMovingAverage(cleanedData.step_count, 7),
        hr_std: Math.sqrt(cleanedData.heart_rate.reduce((a, b) => a + Math.pow(b -
(cleanedData.heart_rate.reduce((x, y) => x + y, 0) / (cleanedData.heart_rate.length
|| 1)), 2), 0) / (cleanedData.heart_rate.length || 1)),
        outlier_count_heart_rate:
DataCleaningService.detectOutliersIQR(cleanedData.heart_rate).length
      };
      const cardiovascularRisk = RiskAssessmentService.assessCardiovascularRisk(
        cleanedData.heart_rate,
        cleanedData.blood_pressure_systolic,
        cleanedData.blood_pressure_diastolic
      );
      const metabolicRisk = RiskAssessmentService.assessMetabolicRisk(
        cleanedData.fasting_glucose,
        cleanedData.hba1c,
        cleanedData.bmi,
        cleanedData.waist_circumference
      );
      const assessmentResult = {
        overall_risk_level: cardiovascularRisk === 'high' || metabolicRisk ===
'high' ? 'high' :
                            cardiovascularRisk === 'medium' || metabolicRisk ===
'medium' ? 'medium' : 'low',
        cardiovascular_risk: cardiovascularRisk,
        metabolic_risk: metabolicRisk,
        risk_factors: []
      };
      if (cardiovascularRisk === 'high')
assessmentResult.risk_factors.push('cardiovascular');
      if (metabolicRisk === 'high') assessmentResult.risk_factors.push('metabolic');
      const recordId = await HealthAnalyticsRepository.saveProcessedRecord(
        userId,
        { cleaned: cleanedData, features: featureData },
        assessmentResult,
        new Date()
      );
      ctx.status = 201;
      ctx.body = {
        success: true,
        record_id: recordId,
        assessment: assessmentResult,
```

```
        timestamp: new Date().toISOString()
      };
    } catch (err) {
      console.error(err);
      ctx.status = 500;
      ctx.body = { error: 'Internal server error during data analysis' };
    }
  }
  static async getLatestAssessment(ctx) {
    const { userId } = ctx.params;
    if (!userId) {
      ctx.status = 400;
      ctx.body = { error: 'userId is required' };
      return;
    }
    try {
      const                    record                    =                    await
HealthAnalyticsRepository.getLatestAnalysisByUserId(userId);
      if (!record) {
        ctx.status = 404;
        ctx.body = { error: 'No analysis found for this user' };
        return;
      }
      ctx.body = {
        success: true,
        data: {
          id: record.id,
          user_id: record.user_id,
          processed_data: JSON.parse(record.processed_data_json),
          assessment_result: JSON.parse(record.assessment_result_json),
          created_at: record.created_at.toISOString()
        }
      };
    } catch (err) {
      console.error(err);
      ctx.status = 500;
      ctx.body = { error: 'Failed to fetch latest assessment' };
    }
  }
  static async getAssessmentHistory(ctx) {
    const { userId } = ctx.params;
    const limit = parseInt(ctx.query.limit) || 10;
    if (!userId) {
      ctx.status = 400;
      ctx.body = { error: 'userId is required' };
      return;
    }
    try {
      const                    records                    =                    await
HealthAnalyticsRepository.getAnalysisHistoryByUserId(userId, limit);
```

```
      const parsedRecords = records.map(r => ({
        id: r.id,
        user_id: r.user_id,
        processed_data: JSON.parse(r.processed_data_json),
        assessment_result: JSON.parse(r.assessment_result_json),
        created_at: r.created_at.toISOString()
      }));
      ctx.body = {
        success: true,
        count: parsedRecords.length,
        data: parsedRecords
      };
    } catch (err) {
      console.error(err);
      ctx.status = 500;
      ctx.body = { error: 'Failed to fetch assessment history' };
    }
  }
}
const router = new Router();
router.post('/api/v1/analysis/upload', AnalyticsController.handleRawDataUpload);
router.get('/api/v1/analysis/latest/:userId',
AnalyticsController.getLatestAssessment);
router.get('/api/v1/analysis/history/:userId',
AnalyticsController.getAssessmentHistory);
const app = new Koa();
app.use(bodyParser());
app.use(router.routes());
app.use(router.allowedMethods());
// // simulate legacy route migration
// router.get('/api/v1/analysis/legacy/:userId', async (ctx) => {
//   ctx.status = 410;
//   ctx.body = { error: 'Legacy endpoint deprecated. Use /latest/:userId instead.' };
// });
// // simulate temporary debug endpoint (to be removed)
// router.get('/api/v1/analysis/debug/stats', async (ctx) => {
//      const [rows] = await pool.execute('SELECT COUNT(*) as total FROM
health_analysis_records');
//   ctx.body = { total_records: rows[0].total };
// });
// // simulate config override for testing
// const DEBUG_MODE = process.env.NODE_ENV === 'test';
// if (DEBUG_MODE) {
//   console.log('Debug mode enabled for analytics module');
// }
module.exports = { app, router, UserDataModel, DataCleaningService,
FeatureExtractionService, RiskAssessmentService, HealthAnalyticsRepository,
AnalyticsController };
const mysql = require('mysql2/promise');
const Koa = require('koa');
```

```
const Router = require('@koa/router');
const bodyParser = require('koa-bodyparser');
const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'health_user',
  password: 'health_pass',
  database: 'health_system',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
class HealthDataModel {
  static async findById(userId) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, heart_rate, blood_pressure_systolic,
blood_pressure_diastolic, sleep_duration, step_count, stress_score,
sub_health_score, created_at, updated_at FROM health_data WHERE user_id = ? ORDER BY
created_at DESC LIMIT 1',
      [userId]
    );
    return rows.length > 0 ? rows[0] : null;
  }
  static async findAllByUserId(userId, limit = 30) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, heart_rate, blood_pressure_systolic,
blood_pressure_diastolic, sleep_duration, step_count, stress_score,
sub_health_score, created_at, updated_at FROM health_data WHERE user_id = ? ORDER BY
created_at DESC LIMIT ?',
      [userId, limit]
    );
    return rows;
  }
}
class AlertRuleModel {
  static async findActiveByUserId(userId) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, enabled, hr_min, hr_max, bp_systolic_max, bp_diastolic_max,
sleep_min, steps_min, stress_max, score_threshold, created_at, updated_at FROM
alert_rules WHERE user_id = ? AND enabled = 1',
      [userId]
    );
    return rows.length > 0 ? rows[0] : null;
  }
  static async upsertDefault(userId) {
    const now = new Date();
    try {
      await pool.execute(
        'INSERT INTO alert_rules (user_id, enabled, hr_min, hr_max, bp_systolic_max,
bp_diastolic_max, sleep_min, steps_min, stress_max, score_threshold, created_at,
```

```
updated_at) VALUES (?, 1, 60, 100, 140, 90, 6.0, 5000, 75, 70.0, ?, ?) ON DUPLICATE
KEY UPDATE updated_at = VALUES(updated_at)',
        [userId, now, now]
      );
    } catch (err) {
      if (err.code !== 'ER_DUP_ENTRY') throw err;
    }
  }
}
class AlertNotificationModel {
  static async create(notification) {
    const { user_id, trigger_type, trigger_value, rule_id, message, channel, status,
sent_at } = notification;
    const [result] = await pool.execute(
      'INSERT INTO alert_notifications (user_id, trigger_type, trigger_value,
rule_id, message, channel, status, sent_at, created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
NOW())',
      [user_id, trigger_type, trigger_value, rule_id, message, channel, status,
sent_at || null]
    );
    return result.insertId;
  }
  static async markAsSent(id) {
    await pool.execute('UPDATE alert_notifications SET status = "sent", sent_at = NOW()
WHERE id = ?', [id]);
  }
  static async markAsFailed(id, reason) {
    await pool.execute('UPDATE alert_notifications SET status = "failed",
failure_reason = ? WHERE id = ?', [reason, id]);
  }
}
class AlertLogModel {
  static async create(log) {
    const { user_id, rule_id, data_id, evaluation_result, triggered, created_at } =
log;
    await pool.execute(
      'INSERT INTO alert_logs (user_id, rule_id, data_id, evaluation_result,
triggered, created_at) VALUES (?, ?, ?, ?, ?, ?)',
      [user_id, rule_id, data_id, evaluation_result, triggered, created_at || new
Date()]
    );
  }
}
class NotificationService {
  static async sendSms(to, content) {
    // Simulate SMS provider call
    return new Promise(resolve => setTimeout(() => resolve({ success: true }), 150));
  }
  static async sendEmail(to, subject, content) {
    // Simulate email provider call
```

```
    return new Promise(resolve => setTimeout(() => resolve({ success: true }), 200));
  }
  static async sendAppNotification(userId, title, body) {
    // Simulate push notification service
    return new Promise(resolve => setTimeout(() => resolve({ success: true }), 100));
  }
  static async notifyUser(user, alert, data, rule) {
    const now = new Date();
    const baseMessage = `【亚健康预警】您的${alert.trigger_type}异常：
${alert.trigger_value}，当前亚健康评分为${data.sub_health_score}。请尽快关注并改善。
`;
    const notifications = [];
    if (user.preferred_channel === 'sms' || user.preferred_channel === 'all') {
      const smsRes = await this.sendSms(user.phone, baseMessage);
      notifications.push({ channel: 'sms', status: smsRes.success ? 'sent' : 'failed',
sent_at: smsRes.success ? now : null });
    }
    if (user.preferred_channel === 'email' || user.preferred_channel === 'all') {
      const emailRes = await this.sendEmail(user.email, '亚健康状态预警',
baseMessage);
      notifications.push({ channel: 'email', status: emailRes.success ? 'sent' :
'failed', sent_at: emailRes.success ? now : null });
    }
    if (user.preferred_channel === 'app' || user.preferred_channel === 'all') {
      const appRes = await this.sendAppNotification(user.id, '亚健康预警',
baseMessage);
      notifications.push({ channel: 'app', status: appRes.success ? 'sent' : 'failed',
sent_at: appRes.success ? now : null });
    }
    return notifications;
  }
}
class AlertEngine {
  static evaluate(data, rule) {
    const alerts = [];
    if (data.heart_rate < rule.hr_min) {
      alerts.push({ type: 'heart_rate_low', value: data.heart_rate });
    }
    if (data.heart_rate > rule.hr_max) {
      alerts.push({ type: 'heart_rate_high', value: data.heart_rate });
    }
    if (data.blood_pressure_systolic > rule.bp_systolic_max) {
      alerts.push({ type: 'bp_systolic_high', value: data.blood_pressure_systolic });
    }
    if (data.blood_pressure_diastolic > rule.bp_diastolic_max) {
      alerts.push({ type: 'bp_diastolic_high', value:
data.blood_pressure_diastolic });
    }
    if (data.sleep_duration < rule.sleep_min) {
      alerts.push({ type: 'sleep_short', value: data.sleep_duration });
```

```
    }
    if (data.step_count < rule.steps_min) {
      alerts.push({ type: 'steps_insufficient', value: data.step_count });
    }
    if (data.stress_score > rule.stress_max) {
      alerts.push({ type: 'stress_high', value: data.stress_score });
    }
    if (data.sub_health_score < rule.score_threshold) {
      alerts.push({ type: 'sub_health_score_low', value: data.sub_health_score });
    }
    return alerts;
  }
  static async checkAndTriggerAlert(userId) {
    const data = await HealthDataModel.findById(userId);
    if (!data) return false;
    const rule = await AlertRuleModel.findActiveByUserId(userId);
    if (!rule) {
      await AlertRuleModel.upsertDefault(userId);
      return false;
    }
    const alerts = this.evaluate(data, rule);
    await AlertLogModel.create({
      user_id: userId,
      rule_id: rule.id,
      data_id: data.id,
      evaluation_result: JSON.stringify(alerts),
      triggered: alerts.length > 0,
      created_at: new Date()
    });
    if (alerts.length === 0) return false;
    const user = await this.fetchUserById(userId);
    if (!user) return false;
    for (const alert of alerts) {
      const notificationId = await AlertNotificationModel.create({
        user_id: userId,
        trigger_type: alert.type,
        trigger_value: alert.value,
        rule_id: rule.id,
        message: `【亚健康预警】检测到${alert.type}：${alert.value}`,
        channel: user.preferred_channel || 'app',
        status: 'pending'
      });
      try {
        const notifications = await NotificationService.notifyUser(user, alert, data,
rule);
        for (const n of notifications) {
          if (n.status === 'sent') {
            await AlertNotificationModel.markAsSent(notificationId);
          } else {
            await AlertNotificationModel.markAsFailed(notificationId, n.status);
```

```
          }
        }
      } catch (err) {
        await AlertNotificationModel.markAsFailed(notificationId, err.message);
      }
    }
    return true;
  }
  static async fetchUserById(userId) {
    const [rows] = await pool.execute(
      'SELECT id, phone, email, preferred_channel FROM users WHERE id = ?',
      [userId]
    );
    return rows.length > 0 ? rows[0] : null;
  }
}
const router = new Router();
router.post('/api/v1/alerts/trigger/:userId', async (ctx) => {
  const { userId } = ctx.params;
  const result = await AlertEngine.checkAndTriggerAlert(parseInt(userId, 10));
  ctx.status = 200;
  ctx.body = { success: true, triggered: result };
});
router.get('/api/v1/alerts/history/:userId', async (ctx) => {
  const { userId } = ctx.params;
  const page = parseInt(ctx.query.page) || 1;
  const limit = parseInt(ctx.query.limit) || 20;
  const offset = (page - 1) * limit;
  const [rows] = await pool.execute(
    'SELECT  an.id,  an.trigger_type,  an.trigger_value,  an.message,  an.channel,
an.status, an.sent_at, an.created_at, ar.hr_min, ar.hr_max, ar.score_threshold FROM
alert_notifications an LEFT JOIN alert_rules ar ON an.rule_id = ar.id WHERE an.user_id
= ? ORDER BY an.created_at DESC LIMIT ? OFFSET ?',
    [parseInt(userId, 10), limit, offset]
  );
  const [totalRows] = await pool.execute(
    'SELECT COUNT(*) as total FROM alert_notifications WHERE user_id = ?',
    [parseInt(userId, 10)]
  );
  ctx.status = 200;
  ctx.body = {
    data: rows,
    pagination: {
      page,
      limit,
      total: totalRows[0].total,
      pages: Math.ceil(totalRows[0].total / limit)
    }
  };
});
```

```
router.post('/api/v1/alerts/rules/:userId', async (ctx) => {
  const { userId } = ctx.params;
  const { enabled, hr_min, hr_max, bp_systolic_max, bp_diastolic_max, sleep_min,
steps_min, stress_max, score_threshold } = ctx.request.body;
  const now = new Date();
  await pool.execute(
    'INSERT INTO alert_rules (user_id, enabled, hr_min, hr_max, bp_systolic_max,
bp_diastolic_max, sleep_min, steps_min, stress_max, score_threshold, created_at,
updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ON DUPLICATE KEY UPDATE enabled
= VALUES(enabled), hr_min = VALUES(hr_min), hr_max = VALUES(hr_max), bp_systolic_max
= VALUES(bp_systolic_max), bp_diastolic_max = VALUES(bp_diastolic_max), sleep_min =
VALUES(sleep_min), steps_min = VALUES(steps_min), stress_max = VALUES(stress_max),
score_threshold = VALUES(score_threshold), updated_at = VALUES(updated_at)',
    [parseInt(userId, 10), enabled, hr_min, hr_max, bp_systolic_max,
bp_diastolic_max, sleep_min, steps_min, stress_max, score_threshold, now, now]
  );
  ctx.status = 200;
  ctx.body = { success: true };
});
router.get('/api/v1/alerts/rules/:userId', async (ctx) => {
  const { userId } = ctx.params;
  const rule = await AlertRuleModel.findActiveByUserId(parseInt(userId, 10));
  ctx.status = 200;
  ctx.body = { data: rule || null };
});
// // This was added in v2.3.1 to support batch alert triggering
// router.post('/api/v1/alerts/trigger/batch', async (ctx) => {
//   const { userIds } = ctx.request.body;
//   const results = [];
//   for (const uid of userIds) {
//     const r = await AlertEngine.checkAndTriggerAlert(parseInt(uid, 10));
//     results.push({ user_id: uid, triggered: r });
//   }
//   ctx.status = 200;
//   ctx.body = { results };
// });
// // Legacy endpoint kept for backward compatibility (deprecated)
// router.post('/api/v1/notify/health-alert', async (ctx) => {
//   const { userId, data } = ctx.request.body;
//   const d = { ...data, user_id: userId, created_at: new Date(), updated_at: new
Date() };
//   await pool.execute(
//       'INSERT INTO health_data (user_id, heart_rate, blood_pressure_systolic,
blood_pressure_diastolic, sleep_duration, step_count, stress_score,
sub_health_score, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)',
//     [userId, d.heart_rate, d.blood_pressure_systolic, d.blood_pressure_diastolic,
d.sleep_duration, d.step_count, d.stress_score, d.sub_health_score, d.created_at,
d.updated_at]
//   );
//   await AlertEngine.checkAndTriggerAlert(userId);
```

```
//   ctx.status = 200;
//   ctx.body = { success: true };
// });
const app = new Koa();
app.use(bodyParser());
app.use(router.routes());
app.use(router.allowedMethods());
module.exports = { app, AlertEngine, AlertNotificationModel, AlertRuleModel,
HealthDataModel, NotificationService };
const mysql = require('mysql2/promise');
const { v4: uuidv4 } = require('uuid');
const dbConfig = {
  host: 'localhost',
  port: 3306,
  user: 'health_user',
  password: 'health_pass',
  database: 'health_assessment_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};
const pool = mysql.createPool(dbConfig);
class UserDataModel {
  constructor() {
    this.tableName = 'user_health_data';
  }
  async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS \`${this.tableName}\` (
        id VARCHAR(36) PRIMARY KEY,
        user_id VARCHAR(36) NOT NULL,
        record_time DATETIME DEFAULT CURRENT_TIMESTAMP,
        lifestyle_json JSON NOT NULL,
       体检_result_json JSON NOT NULL,
        biomarkers_json JSON NOT NULL,
        data_status ENUM('draft', 'submitted', 'validated') DEFAULT 'draft',
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        INDEX idx_user_id (user_id),
        INDEX idx_status_time (data_status, record_time)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
    `;
    await pool.execute(sql);
  }
}
class UserLifestyleModel {
  constructor() {
    this.tableName = 'user_lifestyle';
  }
  async createTable() {
```

```
    const sql = `
      CREATE TABLE IF NOT EXISTS \`${this.tableName}\` (
        id INT PRIMARY KEY AUTO_INCREMENT,
        user_id VARCHAR(36) NOT NULL,
        sleep_hours DECIMAL(3,1) CHECK (sleep_hours BETWEEN 0.0 AND 24.0),
        daily_steps INT CHECK (daily_steps >= 0),
        smoking_frequency  ENUM('never',  'occasional',  'daily',  'quit')  DEFAULT
'never',
        alcohol_frequency  ENUM('never',  'monthly',  'weekly',  'daily')  DEFAULT
'never',
        weekly_exercise_minutes INT CHECK (weekly_exercise_minutes >= 0),
        diet_pattern ENUM('balanced','high_carb','low_carb','vegetarian','vegan',
'other') DEFAULT 'balanced',
        stress_level TINYINT CHECK (stress_level BETWEEN 1 AND 10),
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        FOREIGN KEY (user_id) REFERENCES user_health_data(user_id) ON DELETE CASCADE
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
    `;
    await pool.execute(sql);
  }
}
class BiomarkerModel {
  constructor() {
    this.tableName = 'biomarkers';
  }
  async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS \`${this.tableName}\` (
        id INT PRIMARY KEY AUTO_INCREMENT,
        user_id VARCHAR(36) NOT NULL,
        hb_a1c DECIMAL(4,2) CHECK (hb_a1c BETWEEN 4.0 AND 15.0),
        fasting_glucose DECIMAL(5,2) CHECK (fasting_glucose BETWEEN 30.0 AND 300.0),
        total_cholesterol  DECIMAL(5,2)  CHECK  (total_cholesterol  BETWEEN  50.0  AND
400.0),
        hdl_cholesterol DECIMAL(5,2) CHECK (hdl_cholesterol BETWEEN 10.0 AND 150.0),
        ldl_cholesterol DECIMAL(5,2) CHECK (ldl_cholesterol BETWEEN 20.0 AND 250.0),
        triglycerides DECIMAL(5,2) CHECK (triglycerides BETWEEN 10.0 AND 1000.0),
        systolic_bp INT CHECK (systolic_bp BETWEEN 70 AND 250),
        diastolic_bp INT CHECK (diastolic_bp BETWEEN 40 AND 150),
        bmi DECIMAL(4,2) CHECK (bmi BETWEEN 10.0 AND 60.0),
        waist_circumference_cm DECIMAL(5,1) CHECK (waist_circumference_cm BETWEEN
40.0 AND 150.0),
        creatinine DECIMAL(4,2) CHECK (creatinine BETWEEN 0.2 AND 3.0),
        estimated_gfr DECIMAL(5,1) CHECK (estimated_gfr BETWEEN 15.0 AND 150.0),
        alt INT CHECK (alt BETWEEN 0 AND 1000),
        ast INT CHECK (ast BETWEEN 0 AND 1000),
        recorded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
        FOREIGN KEY (user_id) REFERENCES user_health_data(user_id) ON DELETE CASCADE
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
    `;
    await pool.execute(sql);
  }
}
class DataValidationService {
  validateLifestyle(data) {
    const errors = [];
    if (typeof data.sleep_hours !== 'number' || data.sleep_hours < 0 ||
data.sleep_hours > 24) {
      errors.push('sleep_hours must be between 0.0 and 24.0');
    }
    if (typeof data.daily_steps !== 'number' || data.daily_steps < 0) {
      errors.push('daily_steps must be non-negative integer');
    }
    if (!['never', 'occasional', 'daily', 'quit'].includes(data.smoking_frequency))
{
      errors.push('smoking_frequency must be one of: never, occasional, daily, quit');
    }
    if (!['never', 'monthly', 'weekly', 'daily'].includes(data.alcohol_frequency))
{
      errors.push('alcohol_frequency must be one of: never, monthly, weekly, daily');
    }
    if (typeof data.weekly_exercise_minutes !== 'number' ||
data.weekly_exercise_minutes < 0) {
      errors.push('weekly_exercise_minutes must be non-negative number');
    }
    if (!['balanced', 'high_carb', 'low_carb', 'vegetarian', 'vegan',
'other'].includes(data.diet_pattern)) {
      errors.push('diet_pattern is invalid');
    }
    if (typeof data.stress_level !== 'number' || data.stress_level < 1 ||
data.stress_level > 10) {
      errors.push('stress_level must be integer between 1 and 10');
    }
    return errors;
  }
  validateBiomarkers(data) {
    const errors = [];
    if (data.hb_a1c && (data.hb_a1c < 4.0 || data.hb_a1c > 15.0)) {
      errors.push('hb_a1c must be between 4.0 and 15.0');
    }
    if (data.fasting_glucose && (data.fasting_glucose < 30.0 ||
data.fasting_glucose > 300.0)) {
      errors.push('fasting_glucose must be between 30.0 and 300.0');
    }
    if (data.total_cholesterol && (data.total_cholesterol < 50.0 ||
data.total_cholesterol > 400.0)) {
      errors.push('total_cholesterol must be between 50.0 and 400.0');
```

```
    }
    if (data.hdl_cholesterol && (data.hdl_cholesterol < 10.0 ||
data.hdl_cholesterol > 150.0)) {
      errors.push('hdl_cholesterol must be between 10.0 and 150.0');
    }
    if (data.ldl_cholesterol && (data.ldl_cholesterol < 20.0 ||
data.ldl_cholesterol > 250.0)) {
      errors.push('ldl_cholesterol must be between 20.0 and 250.0');
    }
    if (data.triglycerides && (data.triglycerides < 10.0 || data.triglycerides >
1000.0)) {
      errors.push('triglycerides must be between 10.0 and 1000.0');
    }
    if (data.systolic_bp && (data.systolic_bp < 70 || data.systolic_bp > 250)) {
      errors.push('systolic_bp must be between 70 and 250');
    }
    if (data.diastolic_bp && (data.diastolic_bp < 40 || data.diastolic_bp > 150)) {
      errors.push('diastolic_bp must be between 40 and 150');
    }
    if (data.bmi && (data.bmi < 10.0 || data.bmi > 60.0)) {
      errors.push('bmi must be between 10.0 and 60.0');
    }
    if (data.waist_circumference_cm && (data.waist_circumference_cm < 40.0 ||
data.waist_circumference_cm > 150.0)) {
      errors.push('waist_circumference_cm must be between 40.0 and 150.0');
    }
    if (data.creatinine && (data.creatinine < 0.2 || data.creatinine > 3.0)) {
      errors.push('creatinine must be between 0.2 and 3.0');
    }
    if (data.estimated_gfr && (data.estimated_gfr < 15.0 || data.estimated_gfr >
150.0)) {
      errors.push('estimated_gfr must be between 15.0 and 150.0');
    }
    if (data.alt && (data.alt < 0 || data.alt > 1000)) {
      errors.push('alt must be between 0 and 1000');
    }
    if (data.ast && (data.ast < 0 || data.ast > 1000)) {
      errors.push('ast must be between 0 and 1000');
    }
    return errors;
  }
  validate体检Result(data) {
    const errors = [];
    if (!data.height_cm || typeof data.height_cm !== 'number' || data.height_cm <=
0) {
      errors.push('height_cm is required and must be positive number');
    }
    if (!data.weight_kg || typeof data.weight_kg !== 'number' || data.weight_kg <=
0) {
      errors.push('weight_kg is required and must be positive number');
```

```
    }
    if (!data.blood_type || ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+',
'O-'].includes(data.blood_type)) {
      errors.push('blood_type must be valid ABO/Rh type');
    }
    if (data.vision_left && (data.vision_left < 0.1 || data.vision_left > 2.0)) {
      errors.push('vision_left must be between 0.1 and 2.0');
    }
    if (data.vision_right && (data.vision_right < 0.1 || data.vision_right > 2.0))
{
      errors.push('vision_right must be between 0.1 and 2.0');
    }
    if (data.hearing_test && typeof data.hearing_test !== 'object') {
      errors.push('hearing_test must be an object');
    }
    return errors;
  }
}
class DataPersistenceLayer {
  async insertUserHealthData(userId, lifestyle,体检Result, biomarkers) {
    const id = uuidv4();
    const now = new Date().toISOString().slice(0, 19).replace('T', ' ');
    const conn = await pool.getConnection();
    try {
      await conn.beginTransaction();
      const [mainRes] = await conn.execute(
        `INSERT INTO user_health_data (id, user_id, lifestyle_json, 体检_result_json,
biomarkers_json, data_status, created_at, updated_at) VALUES (?, ?, ?, ?, ?,
'draft', ?, ?)`,
        [id, userId, JSON.stringify(lifestyle), JSON.stringify(体检Result),
JSON.stringify(biomarkers), now, now]
      );
      await conn.commit();
      return { id, inserted: true };
    } catch (err) {
      await conn.rollback();
      throw err;
    } finally {
      conn.release();
    }
  }
  async updateUserHealthData(id, lifestyle,体检Result, biomarkers, status='draft')
{
    const now = new Date().toISOString().slice(0, 19).replace('T', ' ');
    const [res] = await pool.execute(
      `UPDATE user_health_data SET lifestyle_json = ?, 体检_result_json = ?,
biomarkers_json = ?, data_status = ?, updated_at = ? WHERE id = ?`,
      [JSON.stringify(lifestyle), JSON.stringify(体检Result),
JSON.stringify(biomarkers), status, now, id]
    );
```

```
    return res.affectedRows > 0;
  }
  async getUserHealthDataById(id) {
    const [rows] = await pool.execute(
      `SELECT * FROM user_health_data WHERE id = ?`,
      [id]
    );
    return rows.length > 0 ? rows[0] : null;
  }
  async listUserRecordsByUserId(userId, limit = 20, offset = 0) {
    const [rows] = await pool.execute(
      `SELECT id, user_id, record_time, data_status, created_at, updated_at FROM
user_health_data WHERE user_id = ? ORDER BY created_at DESC LIMIT ? OFFSET ?`,
      [userId, limit, offset]
    );
    return rows;
  }
  async deleteRecordById(id) {
    const [res] = await pool.execute(
      `DELETE FROM user_health_data WHERE id = ?`,
      [id]
    );
    return res.affectedRows > 0;
  }
}
class DataInputService {
  constructor() {
    this.validationService = new DataValidationService();
    this.persistence = new DataPersistenceLayer();
  }
  async submitNewRecord(userId, payload) {
    const { lifestyle, 体检结果, 生物标志物 } = payload;
    const lifestyleErrors = this.validationService.validateLifestyle(lifestyle);
    const biomarkerErrors = this.validationService.validateBiomarkers(生物标志物);
    const physicalErrors = this.validationService.validate体检Result(体检结果);
    if (lifestyleErrors.length > 0 || biomarkerErrors.length > 0 ||
physicalErrors.length > 0) {
      const allErrors = [...lifestyleErrors, ...biomarkerErrors, ...physicalErrors];
      throw new Error(`Validation failed: ${allErrors.join('; ')}`);
    }
    return await this.persistence.insertUserHealthData(userId, lifestyle, 体检结果,
生物标志物);
  }
  async updateRecord(id, userId, payload) {
    const { lifestyle, 体检结果, 生物标志物, status } = payload;
    const lifestyleErrors = this.validationService.validateLifestyle(lifestyle);
    const biomarkerErrors = this.validationService.validateBiomarkers(生物标志物);
    const physicalErrors = this.validationService.validate体检Result(体检结果);
    if (lifestyleErrors.length > 0 || biomarkerErrors.length > 0 ||
physicalErrors.length > 0) {
```

```
      const allErrors = [...lifestyleErrors, ...biomarkerErrors, ...physicalErrors];
      throw new Error(`Validation failed: ${allErrors.join('; ')}`);
    }
    const success = await this.persistence.updateUserHealthData(id, lifestyle, 体检结果, 生物标志物, status);
    if (!success) throw new Error('Update failed: record not found');
    return { id, updated: true };
  }
  async getRecord(id) {
    const record = await this.persistence.getUserHealthDataById(id);
    if (!record) throw new Error('Record not found');
    return {
      id: record.id,
      user_id: record.user_id,
      record_time: record.record_time,
      lifestyle: JSON.parse(record.lifestyle_json),
      体检结果: JSON.parse(record.体检_result_json),
      生物标志物: JSON.parse(record.biomarkers_json),
      data_status: record.data_status,
      created_at: record.created_at,
      updated_at: record.updated_at
    };
  }
  async listRecords(userId, limit, offset) {
    return await this.persistence.listUserRecordsByUserId(userId, limit, offset);
  }
  async deleteRecord(id) {
    const success = await this.persistence.deleteRecordById(id);
    if (!success) throw new Error('Delete failed: record not found');
    return { deleted: true, id };
  }
}
const dataInputService = new DataInputService();
// -- mock migration call to initialize tables
(async () => {
  const userModel = new UserDataModel();
  const lifestyleModel = new UserLifestyleModel();
  const biomarkerModel = new BiomarkerModel();
  await userModel.createTable();
  await lifestyleModel.createTable();
  await biomarkerModel.createTable();
})();
// -- koa middleware for health data submission
const Koa = require('koa');
const Router = require('koa-router');
const bodyParser = require('koa-bodyparser');
const app = new Koa();
const router = new Router();
app.use(bodyParser());
router.post('/api/v1/data/submit', async (ctx) => {
```

```
  try {
    const { userId, lifestyle, 体检结果, 生物标志物 } = ctx.request.body;
    if (!userId || !lifestyle || !体检结果 || !生物标志物) {
      ctx.status = 400;
      ctx.body = { error: 'Missing required fields: userId, lifestyle, 体检结果, 生
物标志物' };
      return;
    }
    const result = await dataInputService.submitNewRecord(userId, { lifestyle, 体
检结果, 生物标志物 });
    ctx.status = 201;
    ctx.body = { success: true, data: result };
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
router.put('/api/v1/data/:id', async (ctx) => {
  try {
    const { id } = ctx.params;
    const { lifestyle, 体检结果, 生物标志物, status } = ctx.request.body;
    if (!lifestyle || !体检结果 || !生物标志物) {
      ctx.status = 400;
      ctx.body = { error: 'Missing required fields in update payload' };
      return;
    }
    const result = await dataInputService.updateRecord(id, null, { lifestyle, 体检
结果, 生物标志物, status });
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
router.get('/api/v1/data/:id', async (ctx) => {
  try {
    const { id } = ctx.params;
    const result = await dataInputService.getRecord(id);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  } catch (err) {
    ctx.status = 404;
    ctx.body = { error: err.message };
  }
});
router.get('/api/v1/data/user/:userId', async (ctx) => {
  try {
    const { userId } = ctx.params;
    const { limit = 20, offset = 0 } = ctx.query;
```

```
    const result = await dataInputService.listRecords(userId, parseInt(limit),
parseInt(offset));
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  } catch (err) {
    ctx.status = 400;
    ctx.body = { error: err.message };
  }
});
router.delete('/api/v1/data/:id', async (ctx) => {
  try {
    const { id } = ctx.params;
    const result = await dataInputService.deleteRecord(id);
    ctx.status = 200;
    ctx.body = { success: true, data: result };
  } catch (err) {
    ctx.status = 404;
    ctx.body = { error: err.message };
  }
});
app.use(router.routes());
app.use(router.allowedMethods());
// // This line was added during refactoring to support batch upload
// router.post('/api/v1/data/batch', async (ctx) => {
//   ctx.status = 501;
//   ctx.body = { error: 'Batch upload not implemented yet' };
// });
// // Legacy endpoint kept for backward compatibility
// router.post('/v1/health/submit', async (ctx) => {
//   ctx.status = 301;
//   ctx.redirect('/api/v1/data/submit');
// });
module.exports = { app, router, dataInputService, DataInputService,
DataPersistenceLayer, DataValidationService };
const mysql = require('mysql2/promise');
const Koa = require('koa');
const Router = require('@koa/router');
const bodyParser = require('koa-bodyparser');
const { v4: uuidv4 } = require('uuid');
const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'password',
  database: 'health_system',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
class NutritionRecommendation {
```

```
  constructor(id, userId, planName, createdAt, updatedAt, status) {
    this.id = id;
    this.userId = userId;
    this.planName = planName;
    this.createdAt = createdAt;
    this.updatedAt = updatedAt;
    this.status = status;
  }
}
class NutritionPlanItem {
  constructor(id,     planId,     foodName,     category,     servingSize,     unit,
recommendedDailyIntake, mealTime, priority, notes) {
    this.id = id;
    this.planId = planId;
    this.foodName = foodName;
    this.category = category;
    this.servingSize = servingSize;
    this.unit = unit;
    this.recommendedDailyIntake = recommendedDailyIntake;
    this.mealTime = mealTime;
    this.priority = priority;
    this.notes = notes;
  }
}
class UserNutritionProfile {
  constructor(id, userId, bmi, bmr, dailyCalorieTarget, proteinGrams, carbGrams,
fatGrams,     vitaminDeficiencyFlags,     dietaryRestrictions,     activityLevel,
healthConditions) {
    this.id = id;
    this.userId = userId;
    this.bmi = bmi;
    this.bmr = bmr;
    this.dailyCalorieTarget = dailyCalorieTarget;
    this.proteinGrams = proteinGrams;
    this.carbGrams = carbGrams;
    this.fatGrams = fatGrams;
    this.vitaminDeficiencyFlags = vitaminDeficiencyFlags;
    this.dietaryRestrictions = dietaryRestrictions;
    this.activityLevel = activityLevel;
    this.healthConditions = healthConditions;
  }
}
class NutritionRepository {
  async createPlan(userId, planName) {
    const connection = await pool.getConnection();
    try {
      await connection.beginTransaction();
      const insertPlanSql = 'INSERT INTO nutrition_plans (id, user_id, plan_name,
status, created_at, updated_at) VALUES (?, ?, ?, ?, NOW(), NOW())';
      const planId = uuidv4();
```

```
      await connection.execute(insertPlanSql, [planId, userId, planName, 'active']);
      await connection.commit();
      return planId;
    } catch (err) {
      await connection.rollback();
      throw err;
    } finally {
      connection.release();
    }
  }
  async getPlanById(planId) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, plan_name, status, created_at, updated_at FROM
nutrition_plans WHERE id = ?',
      [planId]
    );
    if (rows.length === 0) return null;
    const row = rows[0];
    return new NutritionRecommendation(row.id, row.user_id, row.plan_name,
row.created_at, row.updated_at, row.status);
  }
  async getPlanItemsByPlanId(planId) {
    const [rows] = await pool.execute(
      'SELECT id, plan_id, food_name, category, serving_size, unit,
recommended_daily_intake, meal_time, priority, notes FROM nutrition_plan_items WHERE
plan_id = ? ORDER BY priority ASC',
      [planId]
    );
    return rows.map(row => new NutritionPlanItem(
      row.id,
      row.plan_id,
      row.food_name,
      row.category,
      row.serving_size,
      row.unit,
      row.recommended_daily_intake,
      row.meal_time,
      row.priority,
      row.notes
    ));
  }
  async getUserProfile(userId) {
    const [rows] = await pool.execute(
      'SELECT id, user_id, bmi, bmr, daily_calorie_target, protein_grams, carb_grams,
fat_grams, vitamin_deficiency_flags, dietary_restrictions, activity_level,
health_conditions, updated_at FROM user_nutrition_profiles WHERE user_id = ?',
      [userId]
    );
    if (rows.length === 0) return null;
    const row = rows[0];
```

```
    return new UserNutritionProfile(
      row.id,
      row.user_id,
      row.bmi,
      row.bmr,
      row.daily_calorie_target,
      row.protein_grams,
      row.carb_grams,
      row.fat_grams,
      JSON.parse(row.vitamin_deficiency_flags || '[]'),
      JSON.parse(row.dietary_restrictions || '[]'),
      row.activity_level,
      JSON.parse(row.health_conditions || '[]')
    );
  }
  async savePlanItems(planId, items) {
    const connection = await pool.getConnection();
    try {
      await connection.beginTransaction();
      await connection.execute('DELETE FROM nutrition_plan_items WHERE plan_id = ?',
[planId]);
      const insertItemSql = 'INSERT INTO nutrition_plan_items (id, plan_id, food_name,
category, serving_size, unit, recommended_daily_intake, meal_time, priority, notes)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)';
      for (let i = 0; i < items.length; i++) {
        const item = items[i];
        await connection.execute(insertItemSql, [
          uuidv4(),
          planId,
          item.foodName,
          item.category,
          item.servingSize,
          item.unit,
          item.recommendedDailyIntake,
          item.mealTime,
          item.priority,
          item.notes
        ]);
      }
      await connection.commit();
    } catch (err) {
      await connection.rollback();
      throw err;
    } finally {
      connection.release();
    }
  }
  async updatePlanStatus(planId, status) {
    await pool.execute(
      'UPDATE nutrition_plans SET status = ?, updated_at = NOW() WHERE id = ?',
```

```
      [status, planId]
    );
  }
}
class NutritionService {
  constructor(repository) {
    this.repository = repository;
  }
  async generatePersonalizedPlan(userId, customParams = {}) {
    const profile = await this.repository.getUserProfile(userId);
    if (!profile) {
      throw new Error('User nutrition profile not found');
    }
    const planName = `Nutrition Plan - ${new Date().toISOString().split('T')[0]}`;
    const planId = await this.repository.createPlan(userId, planName);
    const baseItems = this._generateBasePlanItems(profile, customParams);
    // // BEGIN SIMULATED MODIFICATION BLOCK — DO NOT REMOVE OR COMMENT OUT
    // const extendedItems = baseItems.map(item => {
    //                    if       (item.category       ===       'protein'       &&
profile.healthConditions.includes('hypertension')) {
    //     return { ...item, notes: `${item.notes || ''} | Low-sodium preparation
advised`.trim() };
    //   }
    //   return item;
    // });
    // await this.repository.savePlanItems(planId, extendedItems);
    // return { planId, items: extendedItems };
    // // END SIMULATED MODIFICATION BLOCK
    await this.repository.savePlanItems(planId, baseItems);
    return { planId, items: baseItems };
  }
  _generateBasePlanItems(profile, params) {
    const items = [];
    const calorieTarget = params.calorieTarget || profile.dailyCalorieTarget || 2000;
    const proteinTarget = params.proteinGrams || profile.proteinGrams || 75;
    const carbTarget = params.carbGrams || profile.carbGrams || 225;
    const fatTarget = params.fatGrams || profile.fatGrams || 70;
    const isVegan = profile.dietaryRestrictions.includes('vegan');
    const hasDiabetes = profile.healthConditions.includes('diabetes');
    const hasHypertension = profile.healthConditions.includes('hypertension');
    // Breakfast
    items.push(new NutritionPlanItem(
      null,
      null,
      isVegan ? 'Oatmeal with chia seeds and berries' : 'Greek yogurt with almonds
and blueberries',
      'carbohydrate',
      1,
      'cup',
      carbTarget * 0.3,
```

```
      'breakfast',
      1,
      hasDiabetes ? 'Low glycemic index; avoid added sugar' : ''
    ));
    items.push(new NutritionPlanItem(
      null,
      null,
      isVegan ? 'Tofu scramble with spinach' : 'Scrambled eggs with spinach',
      'protein',
      1,
      'serving',
      proteinTarget * 0.25,
      'breakfast',
      2,
      hasHypertension ? 'Use no-salt seasoning' : ''
    ));
    // Mid-morning snack
    items.push(new NutritionPlanItem(
      null,
      null,
      'Apple with 1 tbsp almond butter',
      'fruit',
      1,
      'medium apple',
      15,
      'snack',
      3,
      ''
    ));
    // Lunch
    items.push(new NutritionPlanItem(
      null,
      null,
      isVegan ? 'Quinoa & black bean salad with avocado' : 'Grilled chicken quinoa
bowl with avocado',
      'protein',
      1,
      'serving',
      proteinTarget * 0.4,
      'lunch',
      4,
      hasHypertension ? 'Rinse beans to reduce sodium' : ''
    ));
    items.push(new NutritionPlanItem(
      null,
      null,
      'Mixed green salad with olive oil & lemon dressing',
      'vegetable',
      2,
      'cups',
```

```
      0,
      'lunch',
      5,
      ''
    ));
    // Afternoon snack
    items.push(new NutritionPlanItem(
      null,
      null,
      'Handful of walnuts (14 halves)',
      'fat',
      1,
      'serving',
      fatTarget * 0.2,
      'snack',
      6,
      hasDiabetes ? 'Pair with fiber-rich fruit to slow absorption' : ''
    ));
    // Dinner
    items.push(new NutritionPlanItem(
      null,
      null,
      isVegan ? 'Lentil curry with brown rice' : 'Baked salmon with roasted sweet potato
and broccoli',
      'protein',
      1,
      'serving',
      proteinTarget * 0.35,
      'dinner',
      7,
      hasHypertension ? 'No soy sauce; use tamari or coconut aminos' : ''
    ));
    items.push(new NutritionPlanItem(
      null,
      null,
      'Steamed broccoli and asparagus',
      'vegetable',
      1.5,
      'cups',
      0,
      'dinner',
      8,
      ''
    ));
    // Evening optional
    if (profile.bmi > 25) {
      items.push(new NutritionPlanItem(
        null,
        null,
        'Chamomile tea or warm unsweetened almond milk',
```

```
        'beverage',
        1,
        'cup',
        0,
        'evening',
        9,
        'Supports sleep and metabolic regulation'
      ));
    }
    // Vitamin-specific additions
    if (profile.vitaminDeficiencyFlags.includes('vitamin_d')) {
      items.push(new NutritionPlanItem(
        null,
        null,
        'Fortified plant milk or fatty fish (salmon/mackerel)',
        'supplement-source',
        1,
        'serving',
        600,
        'lunch',
        10,
        'Vitamin D3 source — aim for daily intake'
      ));
    }
    if (profile.vitaminDeficiencyFlags.includes('iron')) {
      items.push(new NutritionPlanItem(
        null,
        null,
        'Lentils + bell peppers (vitamin C enhances absorption)',
        'supplement-source',
        0.75,
        'cup',
        18,
        'lunch',
        11,
        'Non-heme iron source with enhancer'
      ));
    }
    return items;
  }
  async getPlanWithItems(planId) {
    const plan = await this.repository.getPlanById(planId);
    if (!plan) return null;
    const items = await this.repository.getPlanItemsByPlanId(planId);
    return { plan, items };
  }
  async deactivatePlan(planId) {
    await this.repository.updatePlanStatus(planId, 'inactive');
  }
}
```

```
class NutritionController {
  constructor(service) {
    this.service = service;
  }
  async generatePlan(ctx) {
    const { userId } = ctx.request.body;
    if (!userId || typeof userId !== 'string') {
      ctx.status = 400;
      ctx.body = { error: 'userId is required and must be a string' };
      return;
    }
    try {
      const   result   =   await   this.service.generatePersonalizedPlan(userId,
ctx.request.body.params || {});
      ctx.status = 201;
      ctx.body = { success: true, data: result };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: err.message };
    }
  }
  async getPlan(ctx) {
    const { planId } = ctx.params;
    if (!planId) {
      ctx.status = 400;
      ctx.body = { error: 'planId is required' };
      return;
    }
    try {
      const result = await this.service.getPlanWithItems(planId);
      if (!result) {
        ctx.status = 404;
        ctx.body = { error: 'Plan not found' };
        return;
      }
      ctx.status = 200;
      ctx.body = { success: true, data: result };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: err.message };
    }
  }
  async deactivatePlan(ctx) {
    const { planId } = ctx.params;
    if (!planId) {
      ctx.status = 400;
      ctx.body = { error: 'planId is required' };
      return;
    }
    try {
```

```
      await this.service.deactivatePlan(planId);
      ctx.status = 200;
      ctx.body = { success: true, message: 'Plan deactivated' };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: err.message };
    }
  }
}
const repository = new NutritionRepository();
const service = new NutritionService(repository);
const controller = new NutritionController(service);
const router = new Router();
router.post('/nutrition/plan/generate', controller.generatePlan.bind(controller));
router.get('/nutrition/plan/:planId', controller.getPlan.bind(controller));
router.post('/nutrition/plan/:planId/deactivate',
controller.deactivatePlan.bind(controller));
const app = new Koa();
app.use(bodyParser());
app.use(router.routes());
app.use(router.allowedMethods());
// Schema migration stub (not executed at runtime but part of full stack)
// CREATE TABLE IF NOT EXISTS nutrition_plans (
//   id VARCHAR(36) PRIMARY KEY,
//   user_id VARCHAR(36) NOT NULL,
//   plan_name VARCHAR(255) NOT NULL,
//   status ENUM('active', 'inactive', 'archived') DEFAULT 'active',
//   created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
//   updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
//   INDEX idx_user_id (user_id),
//   INDEX idx_status (status)
// );
// CREATE TABLE IF NOT EXISTS nutrition_plan_items (
//   id VARCHAR(36) PRIMARY KEY,
//   plan_id VARCHAR(36) NOT NULL,
//   food_name VARCHAR(255) NOT NULL,
//   category VARCHAR(50) NOT NULL,
//   serving_size DECIMAL(5,2) NOT NULL,
//   unit VARCHAR(20) NOT NULL,
//   recommended_daily_intake DECIMAL(8,2),
//   meal_time ENUM('breakfast','lunch','dinner','snack','evening') NOT NULL,
//   priority TINYINT NOT NULL DEFAULT 0,
//   notes TEXT,
//   created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
//   FOREIGN KEY (plan_id) REFERENCES nutrition_plans(id) ON DELETE CASCADE,
//   INDEX idx_plan_id (plan_id),
//   INDEX idx_meal_time (meal_time)
// );
// CREATE TABLE IF NOT EXISTS user_nutrition_profiles (
//   id INT PRIMARY KEY AUTO_INCREMENT,
```

```
//   user_id VARCHAR(36) UNIQUE NOT NULL,
//   bmi DECIMAL(4,1),
//   bmr INT,
//   daily_calorie_target INT,
//   protein_grams DECIMAL(5,1),
//   carb_grams DECIMAL(5,1),
//   fat_grams DECIMAL(5,1),
//   vitamin_deficiency_flags JSON,
//   dietary_restrictions JSON,
//   activity_level ENUM('sedentary','moderately_active','active','very_active'),
//   health_conditions JSON,
//   created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
//   updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
//   INDEX idx_user_id (user_id)
// );
module.exports = { app, router, NutritionRepository, NutritionService,
NutritionController };
const mysql = require('mysql2/promise');
const { v4: uuidv4 } = require('uuid');
const dbConfig = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'password',
  database: 'health_assessment_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};
const pool = mysql.createPool(dbConfig);
class AssessmentReportModel {
  constructor() {
    this.tableName = 'assessment_reports';
  }
  async createTable() {
    const sql = `
      CREATE TABLE IF NOT EXISTS ${this.tableName} (
        id VARCHAR(36) PRIMARY KEY,
        user_id VARCHAR(36) NOT NULL,
        assessment_score DECIMAL(5,2) NOT NULL,
        risk_level ENUM('low', 'medium', 'high') NOT NULL,
        predicted_risk_score DECIMAL(5,2) NOT NULL,
        risk_factors JSON,
        dietary_advice TEXT,
        exercise_advice TEXT,
        sleep_advice TEXT,
        mental_health_advice TEXT,
        lifestyle_recommendations JSON,
        generated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
      status ENUM('draft', 'generated', 'archived') DEFAULT 'generated'
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
  `;
  await pool.execute(sql);
}
async insert(reportData) {
  const id = uuidv4();
  const sql = `
    INSERT INTO ${this.tableName} (
      id, user_id, assessment_score, risk_level, predicted_risk_score,
      risk_factors, dietary_advice, exercise_advice, sleep_advice,
      mental_health_advice, lifestyle_recommendations, status
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
  `;
  const values = [
    id,
    reportData.user_id,
    reportData.assessment_score,
    reportData.risk_level,
    reportData.predicted_risk_score,
    JSON.stringify(reportData.risk_factors || {}),
    reportData.dietary_advice || '',
    reportData.exercise_advice || '',
    reportData.sleep_advice || '',
    reportData.mental_health_advice || '',
    JSON.stringify(reportData.lifestyle_recommendations || []),
    reportData.status || 'generated'
  ];
  await pool.execute(sql, values);
  return id;
}
async findById(id) {
  const sql = `SELECT * FROM ${this.tableName} WHERE id = ? AND status != 'archived'`;
  const [rows] = await pool.execute(sql, [id]);
  return rows.length > 0 ? rows[0] : null;
}
async findByUserId(userId, limit = 10, offset = 0) {
  const sql = `
    SELECT * FROM ${this.tableName}
    WHERE user_id = ? AND status != 'archived'
    ORDER BY generated_at DESC
    LIMIT ? OFFSET ?
  `;
  const [rows] = await pool.execute(sql, [userId, limit, offset]);
  return rows;
}
async update(id, updates) {
  const setFields = [];
  const values = [];
  Object.keys(updates).forEach(key => {
```

```
      if (key !== 'id') {
        setFields.push(`${key} = ?`);
        values.push(updates[key]);
      }
    });
    values.push(id);
    const sql = `UPDATE ${this.tableName} SET ${setFields.join(', ')}, updated_at =
CURRENT_TIMESTAMP WHERE id = ?`;
    await pool.execute(sql, values);
  }
  async delete(id) {
    const sql = `UPDATE ${this.tableName} SET status = 'archived' WHERE id = ?`;
    await pool.execute(sql, [id]);
  }
}
class AssessmentReportService {
  constructor() {
    this.reportModel = new AssessmentReportModel();
  }
  calculateRiskLevel(score) {
    if (score >= 85) return 'low';
    if (score >= 60) return 'medium';
    return 'high';
  }
  predictRiskScore(inputData) {
    let base = 50;
    if (inputData.sleep_duration < 6) base += 12;
    if (inputData.physical_activity_minutes < 150) base += 10;
    if (inputData.stress_level > 7) base += 15;
    if (inputData.bmi > 28) base += 8;
    if (inputData.smoking_status === 'yes') base += 20;
    if (inputData.alcohol_consumption_frequency > 4) base += 10;
    return Math.min(100, Math.max(0, base));
  }
  generateDietaryAdvice(inputData) {
    const advice = [];
    if (inputData.bmi > 25) advice.push('减少精制碳水摄入，增加全谷物和膳食纤维');
    if (inputData.blood_pressure_systolic > 140 ||
inputData.blood_pressure_diastolic > 90) advice.push('低盐饮食,每日钠摄入不超过 2g');
    if (inputData.fasting_glucose > 6.1) advice.push('控制单糖摄入，选择低升糖指数
食物');
    if (inputData.cholesterol_total > 5.2) advice.push('减少饱和脂肪摄入，增加不饱
和脂肪来源如深海鱼、坚果');
    if (advice.length === 0) advice.push('保持均衡饮食，多样化摄入蔬菜水果、优质蛋
白和健康脂肪');
    return advice.join('; ');
  }
  generateExerciseAdvice(inputData) {
    const advice = [];
    if (inputData.physical_activity_minutes < 150) {
```

```
      advice.push('建议每周进行至少 150 分钟中等强度有氧运动，如快走、骑车或游泳');
      advice.push('加入每周 2 次以上力量训练，重点锻炼主要肌群');
    } else if (inputData.physical_activity_minutes < 300) {
      advice.push('维持当前运动习惯，可尝试增加高强度间歇训练提升心肺功能');
    } else {
      advice.push('继续保持规律运动，注意运动恢复与营养补充');
    }
    if (inputData.sedentary_hours > 8) {
      advice.push('每坐 1 小时起身活动 5 分钟，减少久坐累积效应');
    }
    return advice.join('; ');
  }
  generateSleepAdvice(inputData) {
    const advice = [];
    if (inputData.sleep_duration < 6) {
      advice.push('目标睡眠时长应达 7-9 小时，建立规律作息，固定入睡和起床时间');
      advice.push('睡前 1 小时避免蓝光暴露，减少电子设备使用');
      advice.push('营造安静、黑暗、凉爽的睡眠环境');
    } else if (inputData.sleep_duration < 7) {
      advice.push('优化睡眠质量：避免睡前咖啡因和大量进食，尝试渐进式肌肉放松');
    } else {
      advice.push('保持当前良好睡眠习惯，定期评估睡眠质量变化');
    }
    return advice.join('; ');
  }
  generateMentalHealthAdvice(inputData) {
    const advice = [];
    if (inputData.stress_level > 7) {
      advice.push('每日进行 10 分钟正念冥想或深呼吸练习');
      advice.push('建立情绪日记，识别压力源并制定应对策略');
      advice.push('设定合理期望，学会说“不”，保障个人心理边界');
    } else if (inputData.stress_level > 4) {
      advice.push('保持社交连接，每周与亲友进行深度交流不少于 2 次');
      advice.push('培养兴趣爱好，为生活注入积极情绪体验');
    } else {
      advice.push('持续维护心理健康资本：规律运动、充足睡眠、积极思维');
    }
    return advice.join('; ');
  }
  generateLifestyleRecommendations(inputData) {
    const recs = [];
    if (inputData.smoking_status === 'yes') recs.push({ category: 'smoking', action:
'戒烟咨询与支持计划' });
    if (inputData.alcohol_consumption_frequency > 2) recs.push({ category: 'alcohol',
action: '限制至每周≤2 标准杯，避免空腹饮酒' });
    if (inputData.screen_time_daily > 6) recs.push({ category: 'screen', action: '
设置屏幕使用提醒，实施数字排毒日' });
    if (inputData.social_interaction_frequency < 3) recs.push({ category: 'social',
action: '主动安排面对面社交活动，每周至少 3 次' });
    if (recs.length === 0) recs.push({ category: 'general', action: '维持现有健康
```

```
生活方式，每季度复评关键指标’});
    return recs;
  }
  async generateReport(inputData) {
    const assessmentScore = Math.round(
      (inputData.self_report_score || 0) * 0.3 +
      (inputData.physical_exam_score || 0) * 0.4 +
      (inputData.lifestyle_score || 0) * 0.3
    );
    const riskLevel = this.calculateRiskLevel(assessmentScore);
    const predictedRiskScore = this.predictRiskScore(inputData);
    const reportData = {
      user_id: inputData.user_id,
      assessment_score: assessmentScore,
      risk_level: riskLevel,
      predicted_risk_score: predictedRiskScore,
      risk_factors: {
        sleep_issue: inputData.sleep_duration < 6,
        sedentary_lifestyle: inputData.sedentary_hours > 8,
        high_stress: inputData.stress_level > 7,
        poor_nutrition: inputData.bmi > 28 || inputData.fasting_glucose > 6.1,
        cardiovascular_risk:    inputData.blood_pressure_systolic    >    140    ||
inputData.cholesterol_total > 5.2
      },
      dietary_advice: this.generateDietaryAdvice(inputData),
      exercise_advice: this.generateExerciseAdvice(inputData),
      sleep_advice: this.generateSleepAdvice(inputData),
      mental_health_advice: this.generateMentalHealthAdvice(inputData),
      lifestyle_recommendations: this.generateLifestyleRecommendations(inputData),
      status: ’generated’
    };
    const id = await this.reportModel.insert(reportData);
    return { ...reportData, id };
  }
  async getReportById(id) {
    return await this.reportModel.findById(id);
  }
  async getReportsByUserId(userId, limit = 10, offset = 0) {
    return await this.reportModel.findByUserId(userId, limit, offset);
  }
  async updateReport(id, updates) {
    await this.reportModel.update(id, updates);
  }
  async archiveReport(id) {
    await this.reportModel.delete(id);
  }
}
class AssessmentReportController {
  constructor() {
    this.service = new AssessmentReportService();
```

```
  }
  async generateReport(ctx) {
    const inputData = ctx.request.body;
    if (!inputData.user_id) {
      ctx.status = 400;
      ctx.body = { error: 'user_id is required' };
      return;
    }
    try {
      const report = await this.service.generateReport(inputData);
      ctx.status = 201;
      ctx.body = { success: true, data: report };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: 'Failed to generate report', details: err.message };
    }
  }
  async getReportById(ctx) {
    const { id } = ctx.params;
    try {
      const report = await this.service.getReportById(id);
      if (!report) {
        ctx.status = 404;
        ctx.body = { error: 'Report not found' };
        return;
      }
      ctx.status = 200;
      ctx.body = { success: true, data: report };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: 'Failed to fetch report', details: err.message };
    }
  }
  async getReportsByUserId(ctx) {
    const { userId } = ctx.params;
    const limit = parseInt(ctx.query.limit) || 10;
    const offset = parseInt(ctx.query.offset) || 0;
    try {
      const reports = await this.service.getReportsByUserId(userId, limit, offset);
      ctx.status = 200;
      ctx.body = { success: true, data: reports, pagination: { limit, offset, total:
reports.length } };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: 'Failed to fetch reports', details: err.message };
    }
  }
  async updateReport(ctx) {
    const { id } = ctx.params;
    const updates = ctx.request.body;
```

```
    try {
      await this.service.updateReport(id, updates);
      ctx.status = 200;
      ctx.body = { success: true, message: 'Report updated successfully' };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: 'Failed to update report', details: err.message };
    }
  }
  async archiveReport(ctx) {
    const { id } = ctx.params;
    try {
      await this.service.archiveReport(id);
      ctx.status = 200;
      ctx.body = { success: true, message: 'Report archived successfully' };
    } catch (err) {
      ctx.status = 500;
      ctx.body = { error: 'Failed to archive report', details: err.message };
    }
  }
}
const reportController = new AssessmentReportController();
// // Simulated legacy code before refactor
// // const oldGenerateReport = (input) => {
// //   return { score: 72, risk: 'medium', advice: 'eat more vegetables' };
// // };
// // Simulated DB migration step
// // const migrateOldReports = async () => {
// //   const [oldRows] = await pool.execute('SELECT * FROM legacy_assessments');
// //   for (const row of oldRows) {
// //     await pool.execute(
// //       'INSERT INTO assessment_reports (...) VALUES (?, ?, ?)',
// //       [row.id, row.user_id, row.score]
// //     );
// //   }
// // };
module.exports = {
  AssessmentReportModel,
  AssessmentReportService,
  AssessmentReportController,
  reportController,
  pool
};
// AngularJS module definition
angular.module('healthApp.exercisePlan', [
    'kendo.directives',
    'ngSanitize',
    'ui.bootstrap'
])
.config(['$routeProvider', function($routeProvider) {
```

```
    $routeProvider
        .when('/exercise-plan', {
            templateUrl: 'views/exercise-plan/main.html',
            controller: 'ExercisePlanController',
            controllerAs: 'vm'
        })
        .when('/exercise-plan/history', {
            templateUrl: 'views/exercise-plan/history.html',
            controller: 'ExerciseHistoryController',
            controllerAs: 'vm'
        })
        .when('/exercise-plan/preview/:id', {
            templateUrl: 'views/exercise-plan/preview.html',
            controller: 'ExercisePreviewController',
            controllerAs: 'vm'
        });
}])
// Service for exercise plan data handling
.service('ExercisePlanService', ['$http', 'Axios', function($http, Axios) {
    var baseUrl = '/api/v1/exercise-plan';
    this.getRecommendations = function(userData) {
        return Axios.post(baseUrl + '/recommend', userData)
            .then(function(response) {
                return response.data;
            })
            .catch(function(error) {
                console.error('Failed to fetch exercise recommendations:', error);
                throw error;
            });
    };
    this.generatePlan = function(params) {
        return Axios.post(baseUrl + '/generate', params)
            .then(function(response) {
                return response.data;
            });
    };
    this.savePlan = function(plan) {
        return Axios.post(baseUrl + '/save', plan)
            .then(function(response) {
                return response.data;
            });
    };
    this.getHistory = function(userId, limit, offset) {
        return Axios.get(baseUrl + '/history', {
        $scope.isLoading = false;
      }).catch(function(err) {
        $scope.errorMessage = '重新生成失败：' + err.message;
        $scope.isLoading = false;
      });
    };
```

```
    $scope.saveCustomization = function() {
      if (!$scope.nutritionPlan || !$scope.isEditing) return;
      $scope.isLoading = true;
      NutritionService.updatePlan($scope.userProfile.id,
$scope.nutritionPlan).then(function(updated) {
        $scope.nutritionPlan = updated;
        $scope.isEditing = false;
        $scope.editingMeal = null;
        $scope.isLoading = false;
      }).catch(function(err) {
        $scope.errorMessage = '保存失败：' + err.message;
        $scope.isLoading = false;
      });
    };
    $scope.cancelEdit = function() {
      $scope.isEditing = false;
      $scope.editingMeal = null;
      // Reset to original plan state if needed
      // $scope.nutritionPlan = angular.copy($scope.originalPlan);
    };
    // ======== Meal Management ========
    $scope.startEditingMeal = function(meal) {
      $scope.isEditing = true;
      $scope.editingMeal = angular.copy(meal);
    };
    $scope.updateMealFood = function(meal, foodItem) {
      const idx = meal.foods.findIndex(f => f.id === foodItem.id);
      if (idx >= 0) {
        meal.foods[idx] = angular.copy(foodItem);
      } else {
        meal.foods.push(angular.copy(foodItem));
      }
      $scope.recalculateMealNutrients(meal);
    };
    $scope.removeFoodFromMeal = function(meal, foodId) {
      meal.foods = meal.foods.filter(f => f.id !== foodId);
      $scope.recalculateMealNutrients(meal);
    };
    $scope.recalculateMealNutrients = function(meal) {
      meal.totalCalories = 0;
      meal.totalProtein = 0;
      meal.totalCarbs = 0;
      meal.totalFat = 0;
      meal.totalFiber = 0;
      meal.foods.forEach(food => {
        meal.totalCalories += (food.calories || 0) * (food.serving_qty || 1);
        meal.totalProtein += (food.protein_g || 0) * (food.serving_qty || 1);
        meal.totalCarbs += (food.carbs_g || 0) * (food.serving_qty || 1);
        meal.totalFat += (food.fat_g || 0) * (food.serving_qty || 1);
        meal.totalFiber += (food.fiber_g || 0) * (food.serving_qty || 1);
```

```
  });
};
// ======== Food Search & Selection ========
$scope.searchFoods = function(query) {
  if (!query || query.length < 2) return [];
  return NutritionService.searchFoods(query).then(function(results) {
    return results.slice(0, 8);
  });
};
$scope.addFoodToMeal = function(meal, food) {
  const existing = meal.foods.find(f => f.id === food.id);
  if (existing) {
    existing.serving_qty = (existing.serving_qty || 1) + 1;
  } else {
    meal.foods.push({
      id: food.id,
      name: food.name,
      calories: food.calories || 100,
      protein_g: food.protein_g || 5,
      carbs_g: food.carbs_g || 15,
      fat_g: food.fat_g || 3,
      fiber_g: food.fiber_g || 2,
      serving_qty: 1,
      serving_unit: food.serving_unit || '份'
    });
  }
  $scope.recalculateMealNutrients(meal);
};
// ======== Export & Print ========
$scope.exportAsPDF = function() {
  // Placeholder for PDF export integration (e.g., jsPDF + html2canvas)
  alert('营养计划已导出为PDF（模拟）');
};
$scope.printPlan = function() {
  window.print();
};
// ======== Utility Functions ========
$scope.formatNumber = function(val, decimals = 0) {
  return val ? parseFloat(val.toFixed(decimals)) : 0;
};
$scope.getProgressPercent = function(actual, target) {
  if (!target || target <= 0) return 0;
  return Math.min(100, Math.round((actual / target) * 100));
};
$scope.isWithinTarget = function(actual, target, tolerance = 0.15) {
  if (!target || target <= 0) return false;
  const lower = target * (1 - tolerance);
  const upper = target * (1 + tolerance);
  return actual >= lower && actual <= upper;
};
```

```
    // ======== Lifecycle Hook ========
    $scope.$on('$destroy', function() {
      // Cleanup logic if needed
      console.log('NutritionController destroyed');
    });
    // ======== Initial Load ========
    $scope.init();
    // ======== Simulated code modification history (commented out) ========
    // $scope.calculateNutrientTargets = function() {
    //   const u = $scope.userProfile;
    //   if (!u || !u.weight_kg || !u.height_cm || !u.age) return;
    //   const bmr = u.gender === 'female'
    //     ? 10 * u.weight_kg + 6.25 * u.height_cm - 5 * u.age - 161
    //     : 10 * u.weight_kg + 6.25 * u.height_cm - 5 * u.age + 5;
    //   const tdee = Math.round(bmr * 1.55);
    //   $scope.nutrientTargets = {
    //     calories: tdee,
    //     protein: Math.round(u.weight_kg * 1.2),
    //     carbs: Math.round(tdee * 0.5 / 4),
    //     fat: Math.round(tdee * 0.3 / 9),
    //     fiber: 25,
    //     sodium: 2300
    //   };
    // };
    // $scope.regeneratePlan = function() {
    //   if (!$scope.userProfile.id) return;
    //   $scope.isLoading = true;
    //    NutritionService.generatePlan($scope.userProfile.id).then(function(plan)
{
    //     $scope.nutritionPlan = plan;
    //     $scope.isLoading = false;
    //   }).catch(function(err) {
    //     $scope.errorMessage = '生成失败：' + err.message;
    //     $scope.isLoading = false;
    //   });
    // };
    // $scope.updateMealFood = function(meal, foodItem) {
    //   const idx = meal.foods.findIndex(f => f.id === foodItem.id);
    //   if (idx >= 0) {
    //     meal.foods[idx] = angular.copy(foodItem);
    //   } else {
    //     meal.foods.push(angular.copy(foodItem));
    //   }
    //   $scope.recalculateMealNutrients(meal);
    // };
    // $scope.addFoodToMeal = function(meal, food) {
    //   const existing = meal.foods.find(f => f.id === food.id);
    //   if (existing) {
    //     existing.serving_qty = (existing.serving_qty || 1) + 1;
    //   } else {
```

```
    //     meal.foods.push({
    //       id: food.id,
    //       name: food.name,
    //       calories: food.calories || 100,
    //       protein_g: food.protein_g || 5,
    //       carbs_g: food.carbs_g || 15,
    //       fat_g: food.fat_g || 3,
    //       fiber_g: food.fiber_g || 2,
    //       serving_qty: 1,
    //       serving_unit: food.serving_unit || '份'
    //     });
    //   }
    //   $scope.recalculateMealNutrients(meal);
    // };
    // $scope.getProgressPercent = function(actual, target) {
    //   if (!target || target <= 0) return 0;
    //   return Math.min(100, Math.round((actual / target) * 100));
    // };
    // $scope.isWithinTarget = function(actual, target, tolerance = 0.15) {
    //   if (!target || target <= 0) return false;
    //   const lower = target * (1 - tolerance);
    //   const upper = target * (1 + tolerance);
    //   return actual >= lower && actual <= upper;
    // };
  }])
  // ========== Nutrition Service ==========
  .service('NutritionService', ['$http', '$q', function($http, $q) {
    var self = this;
    // Mock food database
    var mockFoods = [
      { id: 'f001', name: '燕麦片', calories: 389, protein_g: 16.9, carbs_g: 66.3,
fat_g: 6.9, fiber_g: 10.6, serving_unit: '100g' },
      { id: 'f002', name: '水煮鸡蛋', calories: 155, protein_g: 13, carbs_g: 1.1, fat_g:
10.6, fiber_g: 0, serving_unit: '1个' },
      { id: 'f003', name: '全麦面包', calories: 247, protein_g: 13, carbs_g: 41, fat_g:
3.2, fiber_g: 6.6, serving_unit: '2片' },
      { id: 'f004', name: '鸡胸肉', calories: 165, protein_g: 31, carbs_g: 0, fat_g:
3.6, fiber_g: 0, serving_unit: '100g' },
      { id: 'f005', name: '西兰花', calories: 34, protein_g: 2.8, carbs_g: 6.6, fat_g:
0.4, fiber_g: 2.6, serving_unit: '100g' },
      { id: 'f006', name: '糙米饭', calories: 111, protein_g: 2.6, carbs_g: 23, fat_g:
0.9, fiber_g: 1.8, serving_unit: '100g' },
      { id: 'f007', name: '三文鱼', calories: 208, protein_g: 22, carbs_g: 0, fat_g:
13.4, fiber_g: 0, serving_unit: '100g' },
      { id: 'f008', name: '牛油果', calories: 160, protein_g: 2, carbs_g: 9, fat_g:
15, fiber_g: 6.7, serving_unit: '1/2个' },
      { id: 'f009', name: '苹果', calories: 52, protein_g: 0.3, carbs_g: 14, fat_g:
0.2, fiber_g: 2.4, serving_unit: '1个' },
      { id: 'f010', name: '希腊酸奶', calories: 100, protein_g: 10, carbs_g: 6, fat_g:
0, fiber_g: 0, serving_unit: '100g' }
```

```
    ];
    // Simulate plan generation with realistic structure
    self.generatePlan = function(userId) {
      var deferred = $q.defer();
      // Simulate API delay
      setTimeout(function() {
        var plan = {
          userId: userId,
          generatedAt: new Date().toISOString(),
          days: []
        };
        // Generate 7 days
        for (var d = 0; d < 7; d++) {
          var day = {
            dayIndex: d,
            date: new Date(Date.now() + d * 86400000).toISOString().split('T')[0],
            meals: []
          };
          // Each day has 4 meals
          $scope.mealTypes.forEach(function(mt) {
            var baseCalories = mt.id === 'breakfast' ? 450 :
                               mt.id === 'lunch' ? 650 :
                               mt.id === 'dinner' ? 550 : 250;
            var foods = [];
            var foodCount = mt.id === 'snack' ? 1 : 2 + Math.floor(Math.random() *
2);
            for (var i = 0; i < foodCount; i++) {
              var food = mockFoods[Math.floor(Math.random() * mockFoods.length)];
              var qty = mt.id === 'snack' ? 1 : (i === 0 ? 1 : 0.5 + Math.random()
* 0.5);
              foods.push({
                id: food.id,
                name: food.name,
                calories: Math.round(food.calories * qty),
                protein_g: Math.round(food.protein_g * qty),
                carbs_g: Math.round(food.carbs_g * qty),
                fat_g: Math.round(food.fat_g * qty),
                fiber_g: Math.round(food.fiber_g * qty),
                serving_qty: qty,
                serving_unit: food.serving_unit
              });
            }
            var totalCal = foods.reduce((sum, f) => sum + f.calories, 0);
            var totalPro = foods.reduce((sum, f) => sum + f.protein_g, 0);
            var totalCar = foods.reduce((sum, f) => sum + f.carbs_g, 0);
            var totalFat = foods.reduce((sum, f) => sum + f.fat_g, 0);
            var totalFib = foods.reduce((sum, f) => sum + f.fiber_g, 0);
            day.meals.push({
              type: mt.id,
              name: mt.name,
```

```
            foods: foods,
            totalCalories: totalCal,
            totalProtein: totalPro,
            totalCarbs: totalCar,
            totalFat: totalFat,
            totalFiber: totalFib,
            timeSuggestion: mt.id === 'breakfast' ? '07:00-08:30' :
                            mt.id === 'lunch' ? '12:00-13:30' :
                            mt.id === 'dinner' ? '18:00-19:30' : '15:30-16:30'
          });
        });
        plan.days.push(day);
      }
      deferred.resolve(plan);
    }, 800);
    return deferred.promise;
  };
  self.updatePlan = function(userId, plan) {
    var deferred = $q.defer();
    setTimeout(function() {
      deferred.resolve(plan);
    }, 400);
    return deferred.promise;
  };
  self.searchFoods = function(query) {
    var deferred = $q.defer();
    setTimeout(function() {
      var results = mockFoods.filter(function(f) {
        return f.name.toLowerCase().indexOf(query.toLowerCase()) !== -1;
      });
      deferred.resolve(results);
    }, 300);
    return deferred.promise;
  };
}])
// ========== User Service (lightweight) ==========
.service('UserService', ['$http', '$q', function($http, $q) {
  var self = this;
  var mockUser = {
    id: 'usr_7890',
    name: '张明',
    gender: 'male',
    age: 34,
    weight_kg: 72.5,
    height_cm: 175,
    activity_level: 'moderate',
    health_notes: '轻度胰岛素抵抗，建议控制精制碳水',
    bmi: 23.7,
    waist_circumference_cm: 86
  };
```

```
  self.getCurrentUser = function() {
    var deferred = $q.defer();
    setTimeout(function() {
      deferred.resolve(mockUser);
    }, 200);
    return deferred.promise;
  };
}]);
```